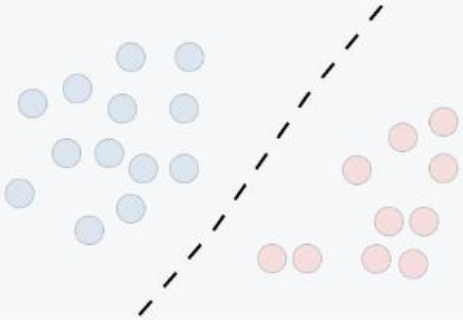
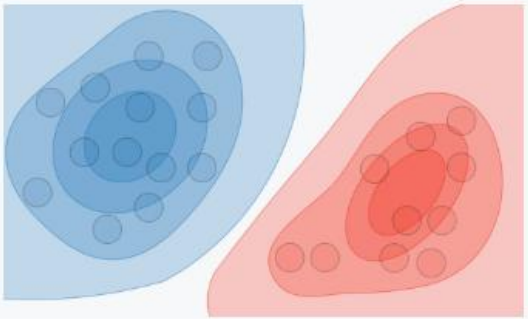
深度生成模型

模型范式

	Discriminative model	Generative model
Goal	Directly estimate $P(y\|x)$	Estimate $P(x\|y)$ to then deduce $P(y\|x)$
What's learned	Decision boundary	Probability distributions of the data
Illustration		
Examples	Regressions, SVMs	GDA, Naive Bayes

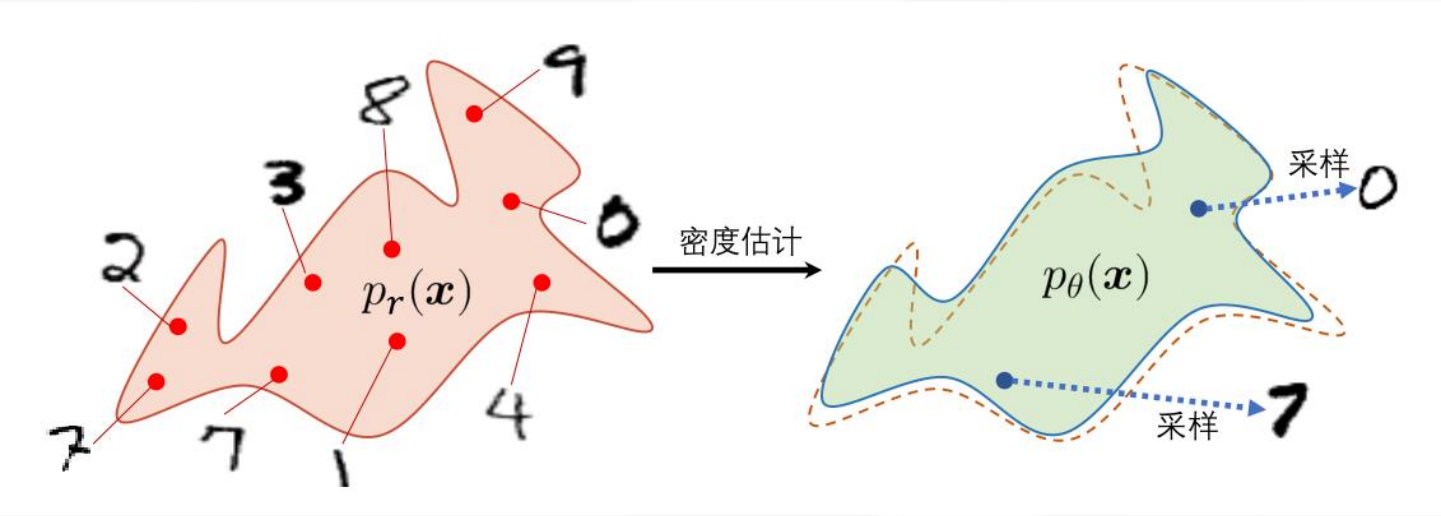
单纯建模$p(y|x)$

建模$p(x,y)$

生成模型

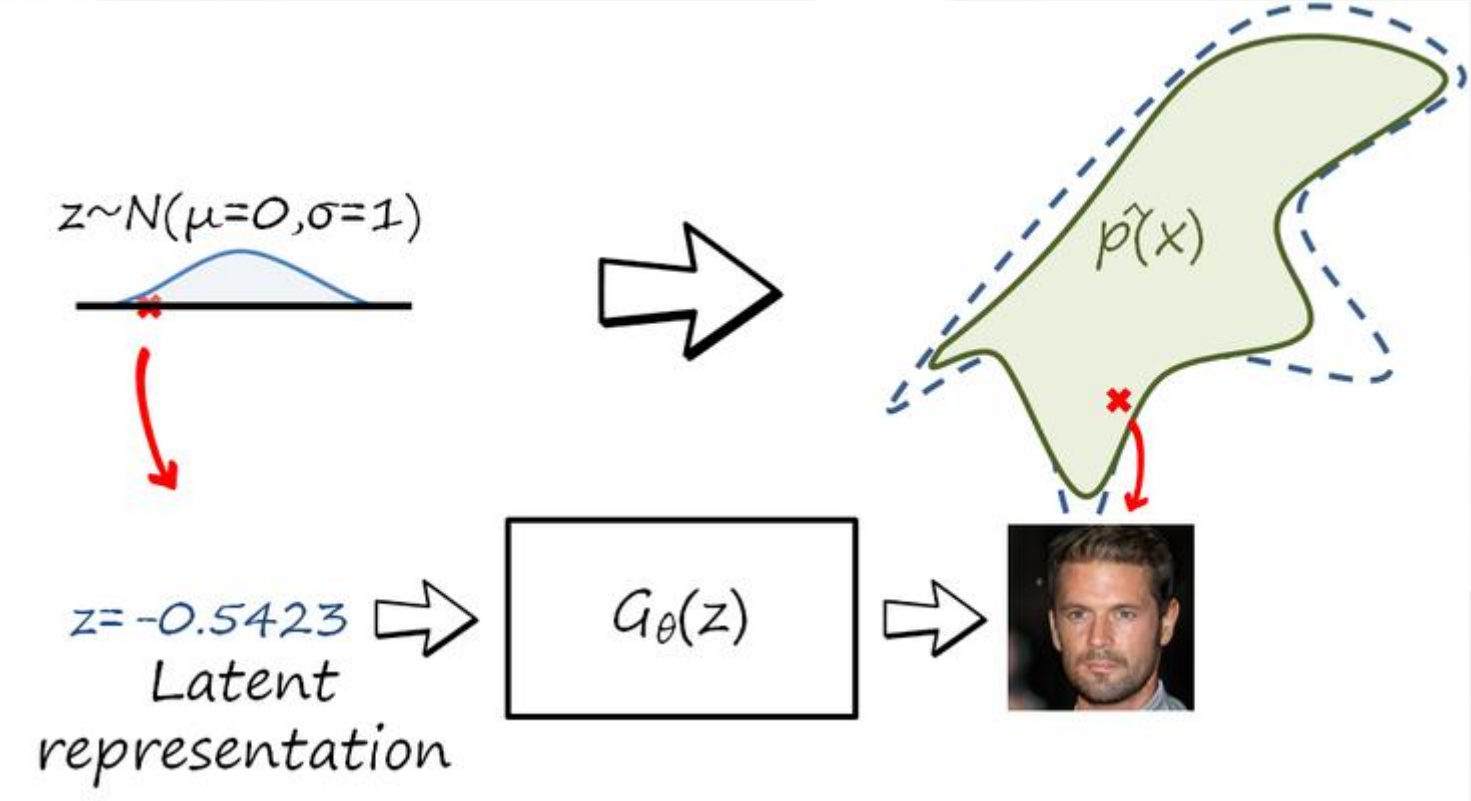
生成模型：一系列用于随机生成可观测数据的模型

- 生成模型包含两个步骤：
  - 密度估计
  - 采样



生成模型

- 生成数据的另一种思路



深度生成模型

深度生成模型：是利用神经网络构建生成模型。

- 变分自编码器（Variational Autoencoder，VAE）
  [Kingma and Welling, 2013, Rezende et al., 2014]
- 生成对抗网络（Generative Adversarial Network，GAN）
  [Goodfellow et al., 2014]
- 扩散模型（Diffusion Model, DM）
  [Jonathan Ho et al., 2020]

显式密度模型和隐式密度模型

显式密度模型

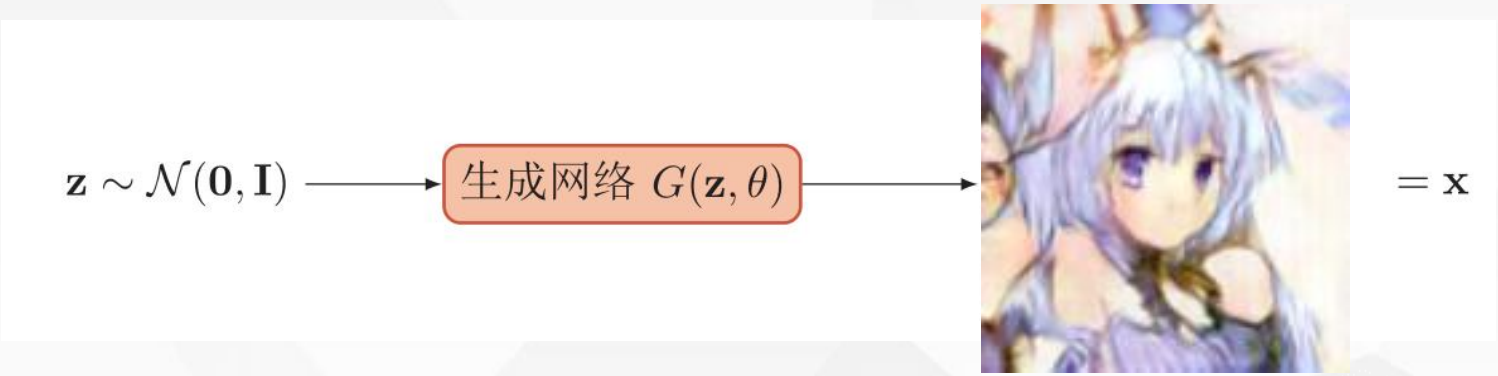
- 显示地构建出样本的密度函数 $p(x|\theta)$, 并通过最大似然估计来求解参数;
- 变分自编码器、深度信念网络

隐式密度模型

- 不显示地估计出数据分布的密度函数
- 但能生成符合数据分布p_{data} (x)的样本
- 无法用最大似然估计

生成网络

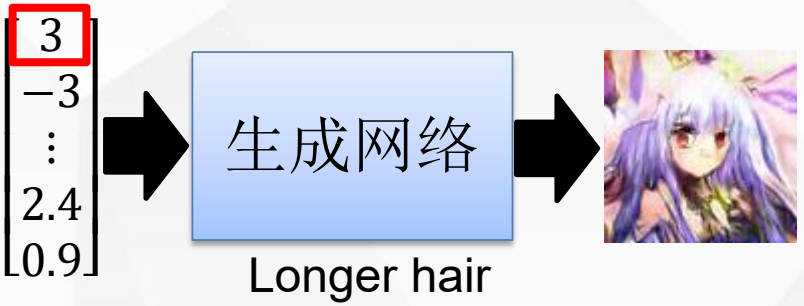
- 生成网络从隐空间（latent space）中随机采样作为输入，其输出结果需要尽量模仿训练集中的真实样本。



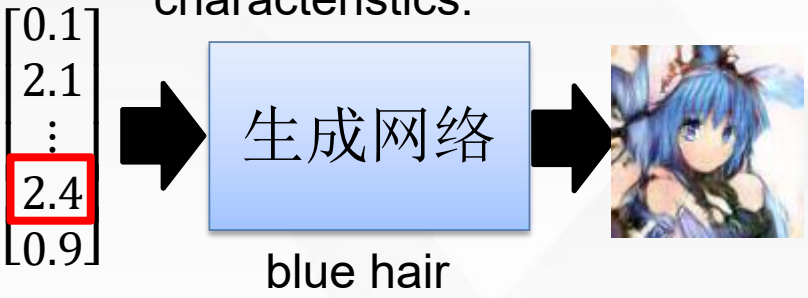
如何学习生成网络？

生成网络示例

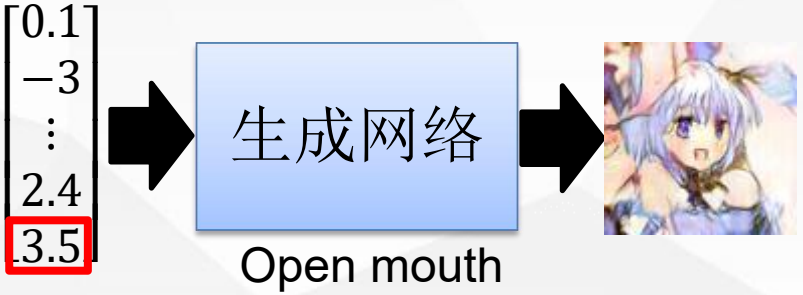
Each dimension of input vector represents some characteristics.



Longer hair



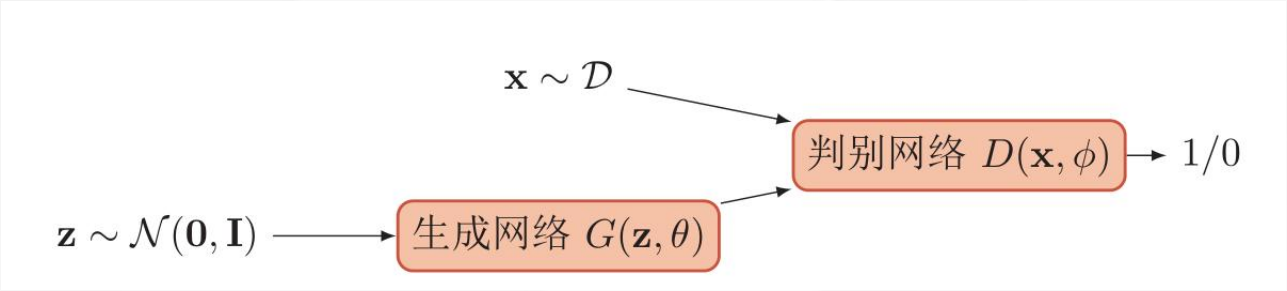
blue hair

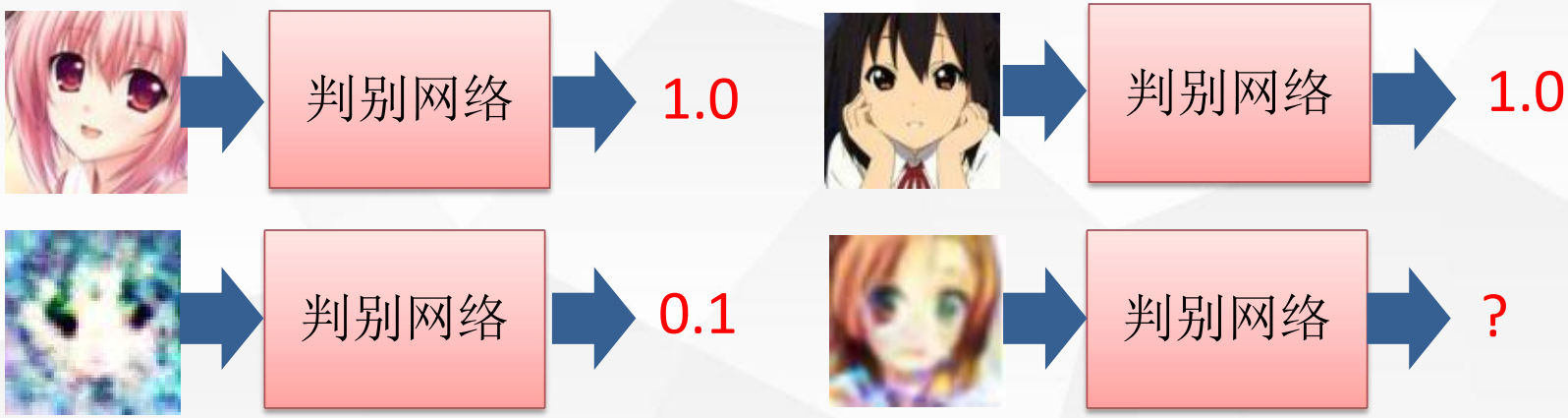


Open mouth

判别网络

- 判别网络的输入则为真实样本或生成网络的输出，其目的是将生成网络的输出从真实样本中尽可能分辨出来。





MinMax Game

对抗训练

- 生成网络要尽可能地欺骗判别网络。
- 判别网络将生成网络生成的样本与真实样本中尽可能区分出来。

两个网络相互对抗、不断调整参数，最终目的是使判别网络无法判断生成网络的输出结果是否真实。

对抗过程

生成网络
(student)

判别网络
(teacher)

生成网络
v1

判别网络
v1

No eyes

生成网络
v2

判别网络
v2

No mouth

生成网络
v3

MinMax Game

判别网络

$$\max_{\phi} \mathbb{E}_{\boldsymbol{x}\sim p_r(\boldsymbol{x})}\Big[\log D(\boldsymbol{x};\phi)\Big] + \mathbb{E}_{\boldsymbol{z}\sim p(\boldsymbol{z})}\Big[\log(1 - D\Big(G(\boldsymbol{z};\theta);\phi\Big)\Big]$$

生成网络

$$\max_{\theta} \Big(\mathbb{E}_{\boldsymbol{z}\sim p(\boldsymbol{z})}\Big[\log D\Big(G(\boldsymbol{z};\theta);\phi\Big)\Big]\Big)$$

判别网络

$$\min_{\theta}\max_{\phi} \Big(\mathbb{E}_{\boldsymbol{x}\sim p_r(\boldsymbol{x})}\Big[\log D(\boldsymbol{x};\phi)\Big] + \mathbb{E}_{\boldsymbol{z}\sim p(\boldsymbol{z})}\Big[\log(1 - D\Big(G(\boldsymbol{z};\theta);\phi\Big)\Big]\Big)$$

训练过程

算法 13.1 生成对抗网络的训练过程

输入: 训练集 $\mathcal{D}$，对抗训练迭代次数 T，每次判别网络的训练迭代次数 K，小批量样本数量 M

随机初始化 θ, ϕ;

for $t \leftarrow 1$ **to** T **do**

// 训练判别网络 $D(\boldsymbol{x}; \phi)$

for $k \leftarrow 1$ **to** K **do**

// 采集小批量训练样本

从训练集 $\mathcal{D}$ 中采集 M 个样本 $\{\boldsymbol{x}^{(m)}\}, 1 \leq m \leq M$;

从分布 $\mathcal{N}(\mathbf{0}, \boldsymbol{I})$ 中采集 M 个样本 $\{\boldsymbol{z}^{(m)}\}, 1 \leq m \leq M$;

使用随机梯度上升更新 ϕ，梯度为

$$\frac{\partial}{\partial \phi}\left[\frac{1}{M}\sum_{m=1}^{M}\left(\log D(\boldsymbol{x}^{(m)}; \phi) + \log\left(1 - D(G(\boldsymbol{z}^{(m)}; \theta); \phi)\right)\right)\right];$$

end

// 训练生成网络 $G(\boldsymbol{z}; \theta)$

从分布 $\mathcal{N}(\mathbf{0}, \boldsymbol{I})$ 中采集 M 个样本 $\{\boldsymbol{z}^{(m)}\}, 1 \leq m \leq M$;

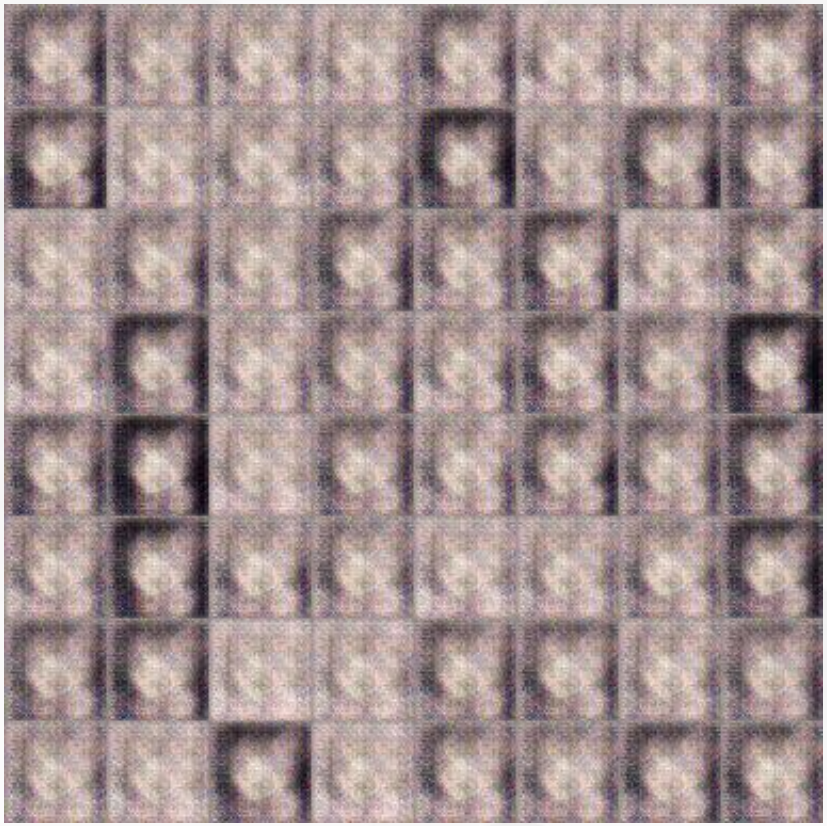
使用随机梯度上升更新 θ，梯度为

$$\frac{\partial}{\partial \theta}\left[\frac{1}{M}\sum_{m=1}^{M} D(G(\boldsymbol{z}^{(m)}; \theta), \phi)\right];$$

end

输出: 生成网络 $G(\boldsymbol{z}; \theta)$

Anime Face Generation



100 updates

1000 updates

Anime Face Generation

2000 updates

5000 updates

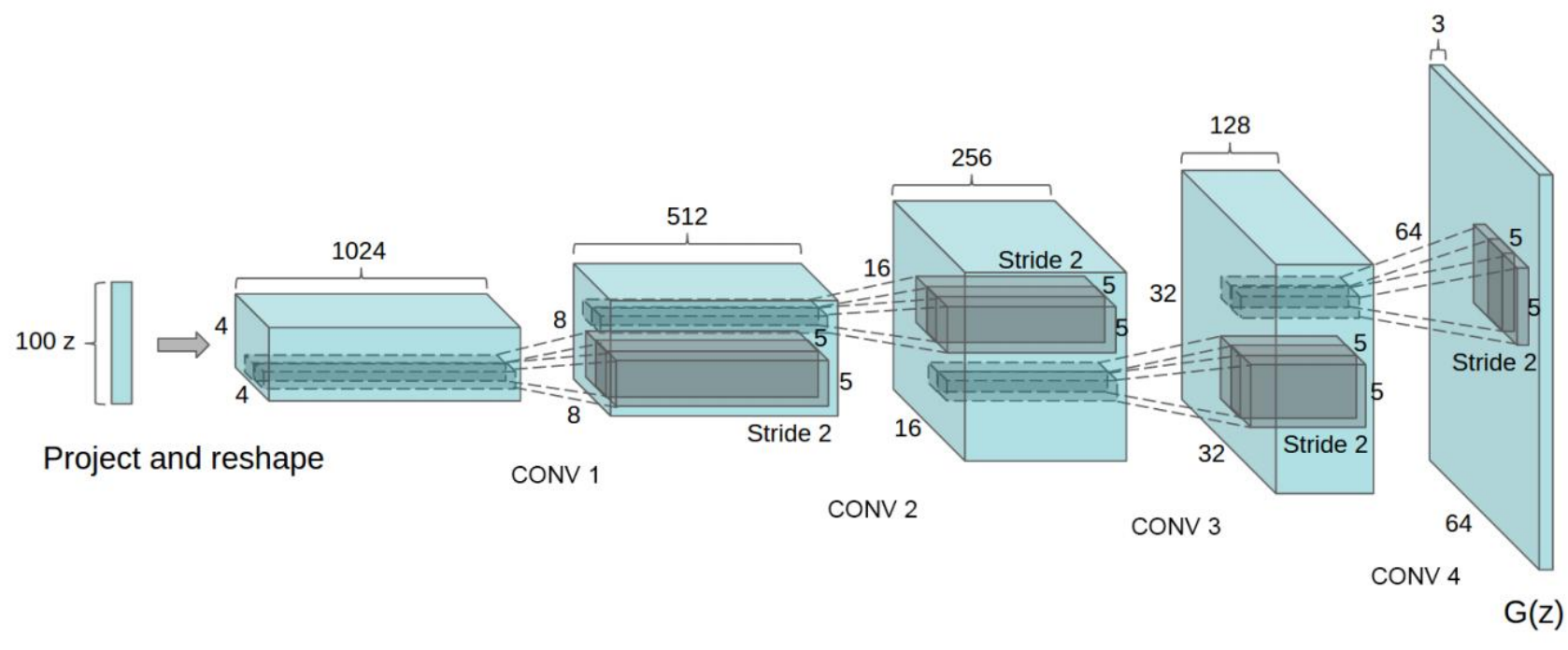
Anime Face Generation

10,000 updates

50,000 updates

一个具体的模型：DCGANs

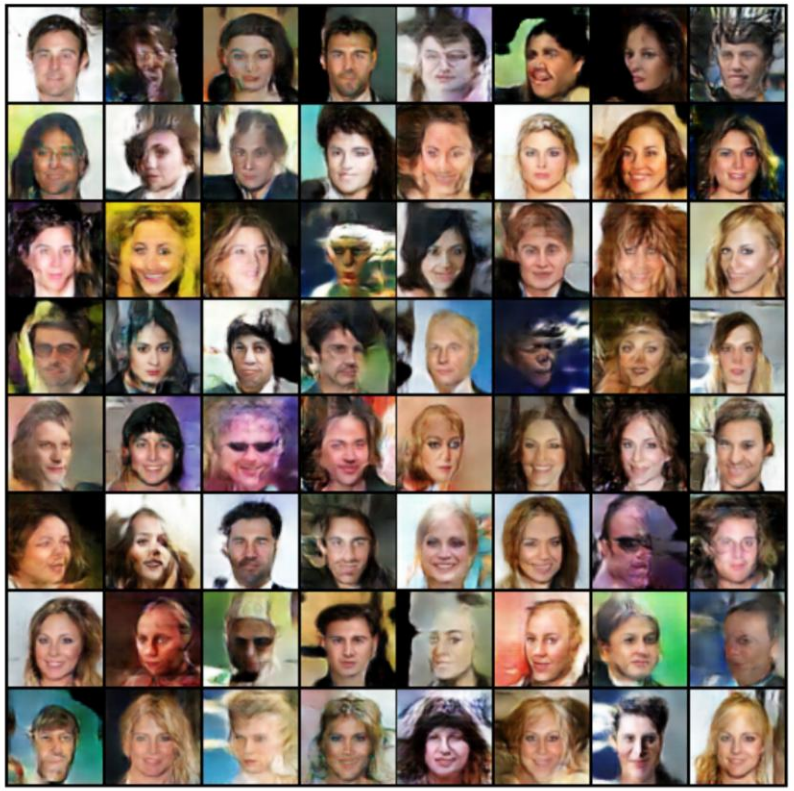
- 判别网络是一个传统的深度卷积网络，但使用了带步长的卷积来实现下采样操作，不用最大汇聚（pooling）操作。
- 生成网络使用一个特殊的深度卷积网络来实现，使用微步卷积来生成64×63大小的图像。



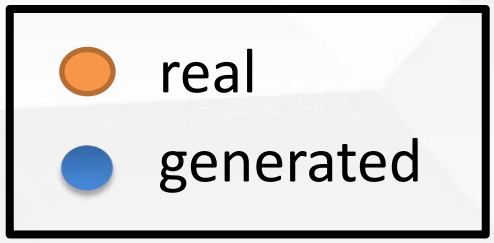
DCGANs

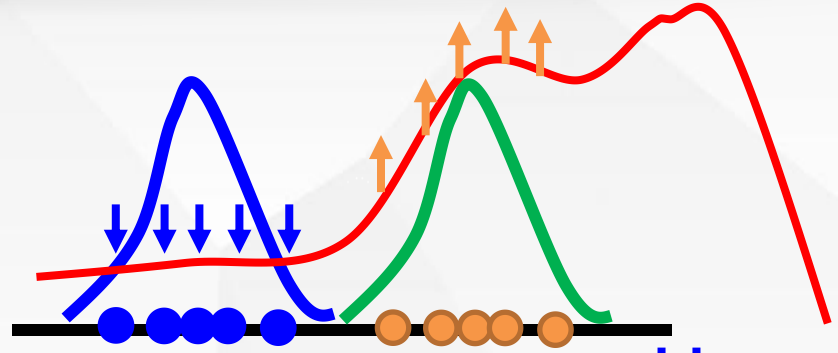
Real Images

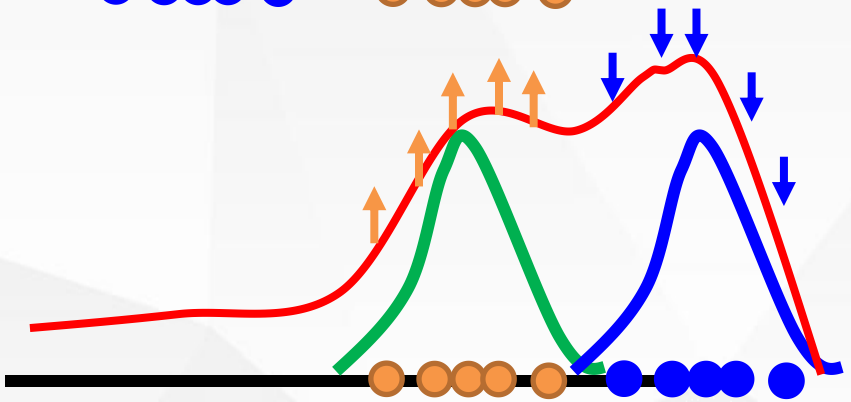
Fake Images



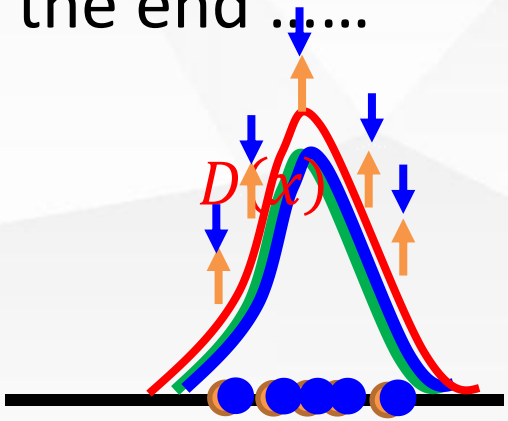
数据分布

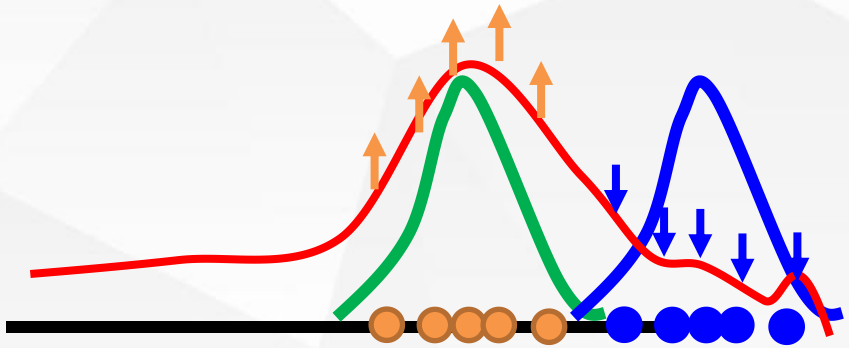






In the end ……





模型分析

- 假设$p_r(x)$和$p_\theta(x)$已知，则最优的判别器为

$$D^\star(\boldsymbol{x}) = \frac{p_r(\boldsymbol{x})}{p_r(\boldsymbol{x}) + p_\theta(\boldsymbol{x})}$$

- 目标函数为

$$\begin{aligned}
\mathcal{L}(G|D^\star) &= \mathbb{E}_{\boldsymbol{x}\sim p_r(\boldsymbol{x})}\Big[\log D^\star(\boldsymbol{x})\Big] + \mathbb{E}_{\boldsymbol{x}\sim p_\theta(\boldsymbol{x})}\Big[\log(1 - D^\star(\boldsymbol{x}))\Big] \\
&= \mathbb{E}_{\boldsymbol{x}\sim p_r(\boldsymbol{x})}\left[\log \frac{p_r(\boldsymbol{x})}{p_r(\boldsymbol{x}) + p_\theta(\boldsymbol{x})}\right] + \mathbb{E}_{\boldsymbol{x}\sim p_\theta(\boldsymbol{x})}\left[\log \frac{p_\theta(\boldsymbol{x})}{p_r(\boldsymbol{x}) + p_\theta(\boldsymbol{x})}\right] \\
&= D_{\mathrm{KL}}\Big(p_r \| p_a\Big) + D_{\mathrm{KL}}\Big(p_\theta \| p_a\Big) - 2\log 2 \\
&= 2D_{\mathrm{JS}}(p_r \| p_\theta) - 2\log 2,
\end{aligned}$$

$$D_{\mathrm{JS}}(p\|q) = \frac{1}{2}D_{\mathrm{KL}}(p\|m) + \frac{1}{2}D_{KL}(q\|m)$$

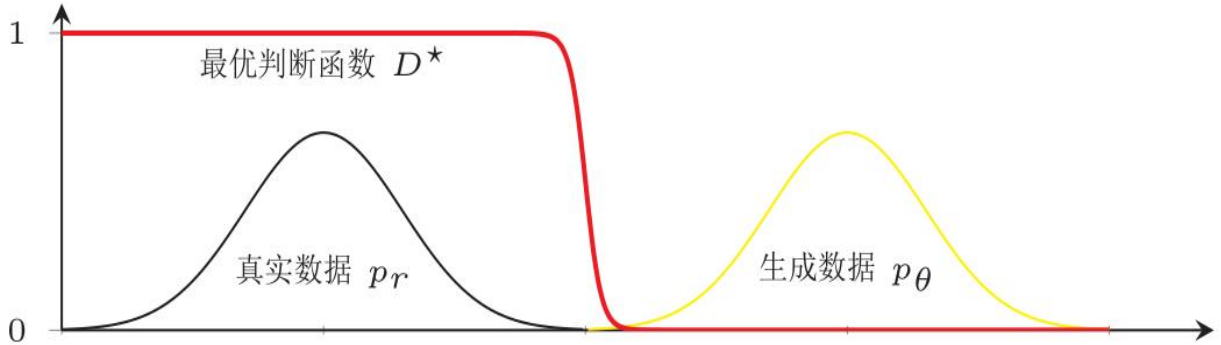
$$m = \tfrac{1}{2}(p+q)$$

不稳定性：生成网络的梯度消失

$$\min_{\theta}\max_{\phi}\left(\mathbb{E}_{\mathbf{x}\sim p_{data}(\mathbf{x})}\left[\log D(\mathbf{x},\phi)\right]+\mathbb{E}_{\mathbf{z}\sim p(\mathbf{z})}\left[\log(1-D\Big(G(\mathbf{z},\theta),\phi\Big)\right]\right)$$

在生成对抗网络中，当判断网络为最优时，生成网络的优化目标是最小化真实分布$p_r(x)$和模型分布$p_\theta(x)$之间的JS散度。
当两个分布相同时，JS散度为0，最优生成网络对应的损失为−2log2。

使用JS散度来训练生成对抗网络的一个问题是当两个分布没有重叠时，它们之间的JS散度恒等于常数log2。对生成网络来说，目标函数关于参数的梯度为0。



$$\frac{\partial\mathcal{L}(G|D^\star)}{\partial\theta}=0.$$

模型坍塌：生成网络的“错误”目标

- 生成网络的目标函数

$$
\begin{aligned}
\mathcal{L}'(G|D^\star) &= \mathbb{E}_{\boldsymbol{x}\sim p_\theta(\boldsymbol{x})}\Big[\log D^\star(\boldsymbol{x})\Big] \\
&= \mathbb{E}_{\boldsymbol{x}\sim p_\theta(\boldsymbol{x})}\left[\log \frac{p_r(\boldsymbol{x})}{p_r(\boldsymbol{x})+p_\theta(\boldsymbol{x})}\cdot\frac{p_\theta(\boldsymbol{x})}{p_\theta(\boldsymbol{x})}\right] \\
&= -\mathbb{E}_{\boldsymbol{x}\sim p_\theta(\boldsymbol{x})}\left[\log \frac{p_\theta(\boldsymbol{x})}{p_r(\boldsymbol{x})}\right] + \mathbb{E}_{\boldsymbol{x}\sim p_\theta(\boldsymbol{x})}\left[\log \frac{p_\theta(\boldsymbol{x})}{p_r(\boldsymbol{x})+p_\theta(\boldsymbol{x})}\right] \\
&= -D_{\mathrm{KL}}\Big(p_\theta\|p_r\Big) + \mathbb{E}_{\boldsymbol{x}\sim p_\theta(\boldsymbol{x})}\Big[\log\big(1-D^\star(\boldsymbol{x})\big)\Big] \\
&= -D_{\mathrm{KL}}\Big(p_\theta\|p_r\Big) + 2D_{\mathrm{JS}}(p_r\|p_\theta) - 2\log 2 - \mathbb{E}_{\boldsymbol{x}\sim p_r(\boldsymbol{x})}\Big[\log D^\star(\boldsymbol{x})\Big]
\end{aligned}
$$

- 其中后两项和生成网络无关，因此

$$
\arg\max_\theta \mathcal{L}'(G|D^\star) = \arg\min_\theta D_{\mathrm{KL}}\Big(p_\theta\|p_r\Big) - 2D_{\mathrm{JS}}(p_r\|p_\theta)
$$

模型坍塌：生成网络的“错误”目标

- 生成网络的目标函数

$$
\begin{aligned}
\mathcal{L}'(G|D^\star) &= \mathbb{E}_{\boldsymbol{x}\sim p_\theta(\boldsymbol{x})}\Big[\log D^\star(\boldsymbol{x})\Big] \\
&= \mathbb{E}_{\boldsymbol{x}\sim p_\theta(\boldsymbol{x})}\left[\log \frac{p_r(\boldsymbol{x})}{p_r(\boldsymbol{x})+p_\theta(\boldsymbol{x})}\cdot\frac{p_\theta(\boldsymbol{x})}{p_\theta(\boldsymbol{x})}\right] \\
&= -\mathbb{E}_{\boldsymbol{x}\sim p_\theta(\boldsymbol{x})}\left[\log \frac{p_\theta(\boldsymbol{x})}{p_r(\boldsymbol{x})}\right] + \mathbb{E}_{\boldsymbol{x}\sim p_\theta(\boldsymbol{x})}\left[\log \frac{p_\theta(\boldsymbol{x})}{p_r(\boldsymbol{x})+p_\theta(\boldsymbol{x})}\right] \\
&= -D_{\mathrm{KL}}\Big(p_\theta\|p_r\Big) + \mathbb{E}_{\boldsymbol{x}\sim p_\theta(\boldsymbol{x})}\Big[\log\big(1-D^\star(\boldsymbol{x})\big)\Big] \\
&= -D_{\mathrm{KL}}\Big(p_\theta\|p_r\Big) + 2D_{\mathrm{JS}}(p_r\|p_\theta) - 2\log 2 - \mathbb{E}_{\boldsymbol{x}\sim p_r(\boldsymbol{x})}\Big[\log D^\star(\boldsymbol{x})\Big]
\end{aligned}
$$

If $p_{data}(x) \to 0$ and $p_g(x) \to 1$, we have $KL(p_g \| p_{data}) \to +\infty$.

If $p_{data}(x) \to 1$ and $p_g(x) \to 0$, we have $KL(p_g \| p_{data}) \to 0$.

前向和逆向KL散度

- KL散度是一种非对称的散度，在计算真实分布$p_r(x)$和模型分布$p_\theta(x)$之间的KL散度时，按照顺序不同，有两种KL散度：

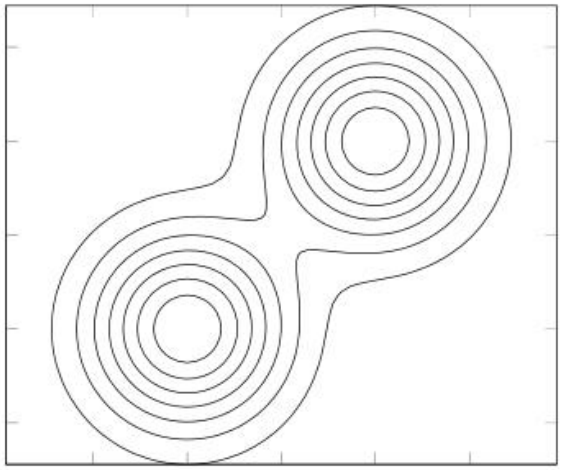
前向KL散度

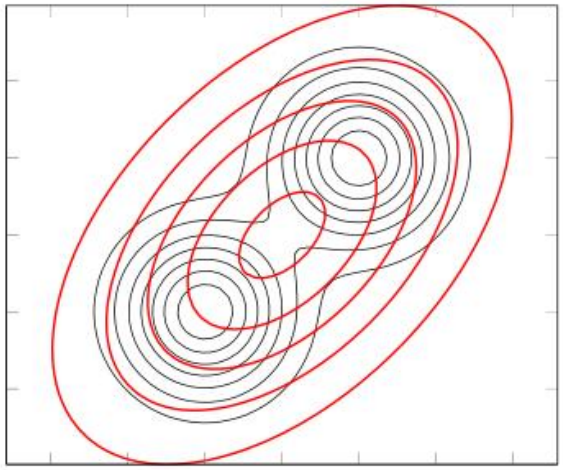
$$D_{\mathrm{KL}}(p_r \| p_\theta) = \int p_r(\boldsymbol{x}) \log \frac{p_r(\boldsymbol{x})}{p_\theta(\boldsymbol{x})} d\boldsymbol{x}$$

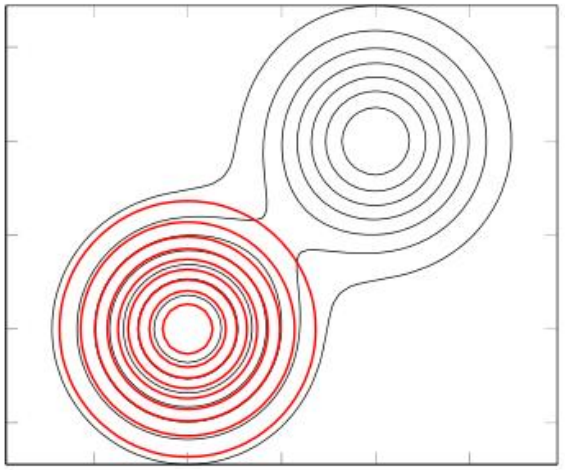
逆向KL散度

$$D_{\mathrm{KL}}(p_\theta \| p_r) = \int p_\theta(\boldsymbol{x}) \log \frac{p_\theta(\boldsymbol{x})}{p_r(\boldsymbol{x})} d\boldsymbol{x}$$

前向和逆向KL散度





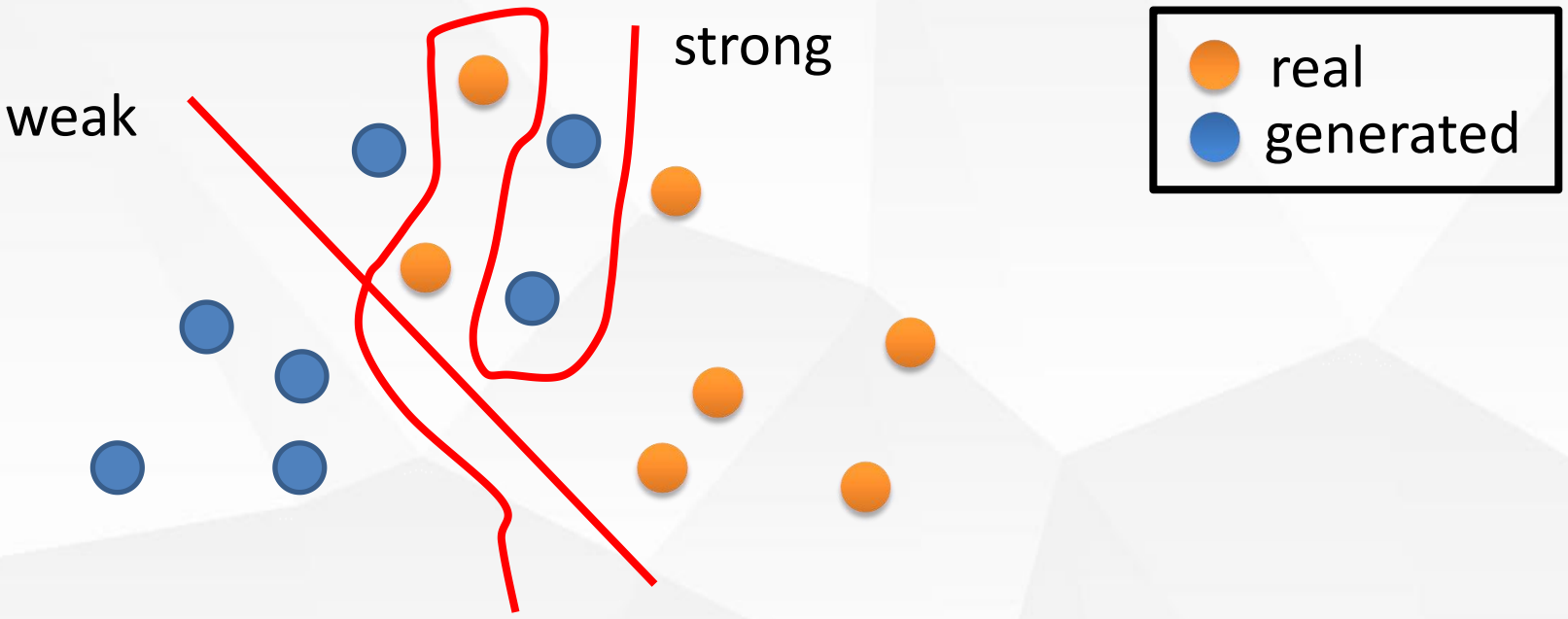


$$D_{\mathrm{KL}}(p_r\|p_\theta) = \int p_r(\boldsymbol{x}) \log \frac{p_r(\boldsymbol{x})}{p_\theta(\boldsymbol{x})} d\boldsymbol{x}$$

$$D_{\mathrm{KL}}(p_\theta\|p_r) = \int p_\theta(\boldsymbol{x}) \log \frac{p_\theta(\boldsymbol{x})}{p_r(\boldsymbol{x})} d\boldsymbol{x}$$

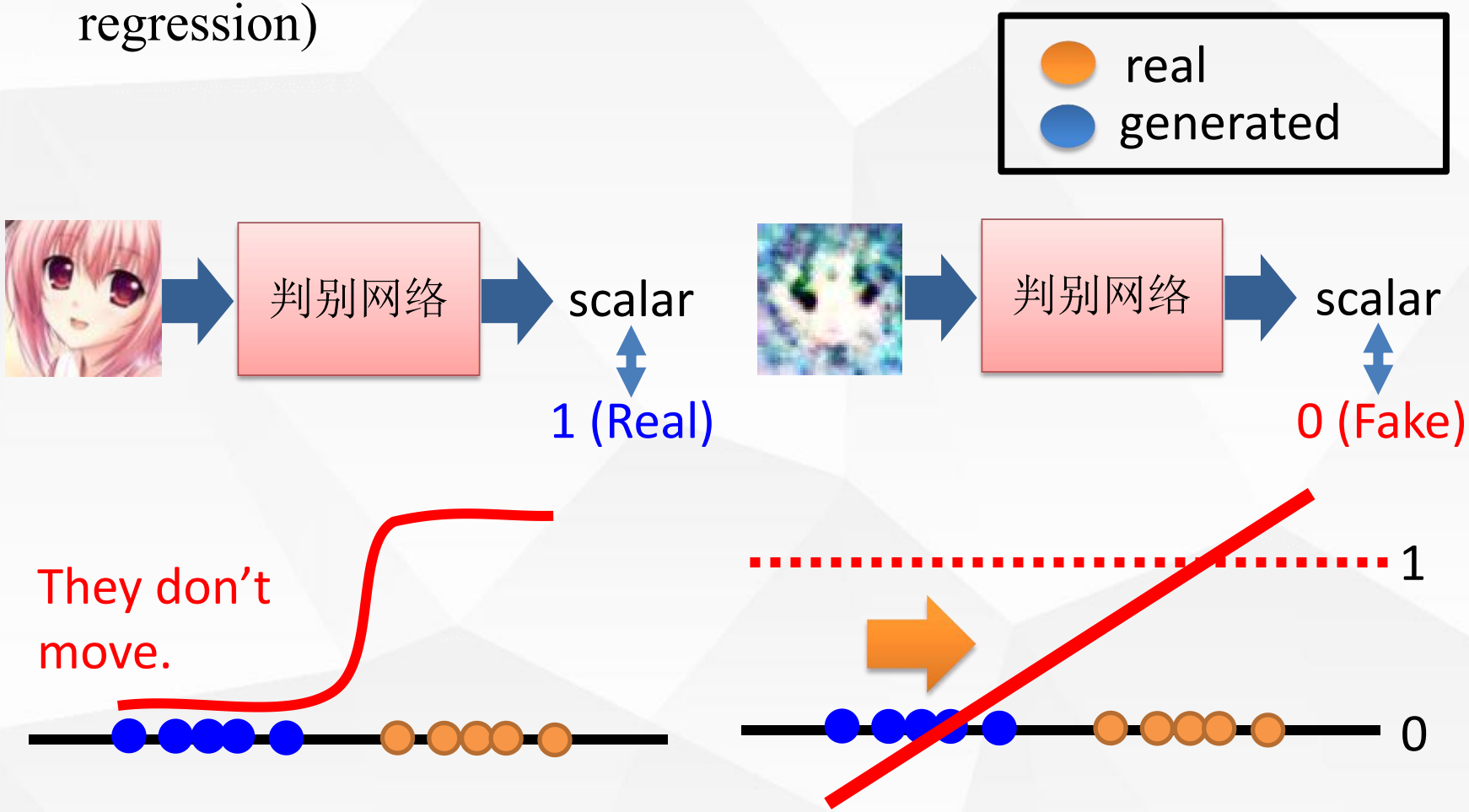
改进

- 弱化判别器
- 使用更好的损失函数



Least Square GAN (LSGAN)

- Replace sigmoid with linear (replace classification with regression)



f-GAN

f-divergences $D_f(P||Q)$

Name	$D_f(P\|Q)$	Generator $f(u)$	$T^*(x)$
Kullback-Leibler	$\int p(x) \log \frac{p(x)}{q(x)} \mathrm{d}x$	$u \log u$	$1 + \log \frac{p(x)}{q(x)}$
Reverse KL	$\int q(x) \log \frac{q(x)}{p(x)} \mathrm{d}x$	$-\log u$	$-\frac{q(x)}{p(x)}$
Pearson χ^2	$\int \frac{(q(x)-p(x))^2}{p(x)} \mathrm{d}x$	$(u-1)^2$	$2(\frac{p(x)}{q(x)} - 1)$
Squared Hellinger	$\int \left(\sqrt{p(x)} - \sqrt{q(x)}\right)^2 \mathrm{d}x$	$\left(\sqrt{u}-1\right)^2$	$(\sqrt{\frac{p(x)}{q(x)}} - 1) \cdot \sqrt{\frac{q(x)}{p(x)}}$
Jensen-Shannon	$\frac{1}{2} \int p(x) \log \frac{2p(x)}{p(x)+q(x)} + q(x) \log \frac{2q(x)}{p(x)+q(x)} \mathrm{d}x$	$-(u+1) \log \frac{1+u}{2} + u \log u$	$\log \frac{2p(x)}{p(x)+q(x)}$
GAN	$\int p(x) \log \frac{2p(x)}{p(x)+q(x)} + q(x) \log \frac{2q(x)}{p(x)+q(x)} \mathrm{d}x - \log(4)$	$u \log u - (u+1) \log(u+1)$	$\log \frac{p(x)}{p(x)+q(x)}$

f-GAN

- 目标函数

$$F(\theta, \omega) = \mathbb{E}_{x \sim P}\left[T_\omega(x)\right] - \mathbb{E}_{x \sim Q_\theta}\left[f^*(T_\omega(x))\right]$$

- f-divergences

Name	Output activation g_f	dom_{f^*}	Conjugate $f^*(t)$	$f'(1)$
Kullback-Leibler (KL)	v	$\mathbb{R}$	$\exp(t-1)$	1
Reverse KL	$-\exp(-v)$	$\mathbb{R}_-$	$-1-\log(-t)$	-1
Pearson χ^2	v	$\mathbb{R}$	$\frac{1}{4}t^2+t$	0
Squared Hellinger	$1-\exp(-v)$	$t<1$	$\frac{t}{1-t}$	0
Jensen-Shannon	$\log(2)-\log(1+\exp(-v))$	$t<\log(2)$	$-\log(2-\exp(t))$	0
GAN	$-\log(1+\exp(-v))$	$\mathbb{R}_-$	$-\log(1-\exp(t))$	$-\log(2)$

Wasserstein距离

- Wasserstein距离用于衡量两个分布之间的距离

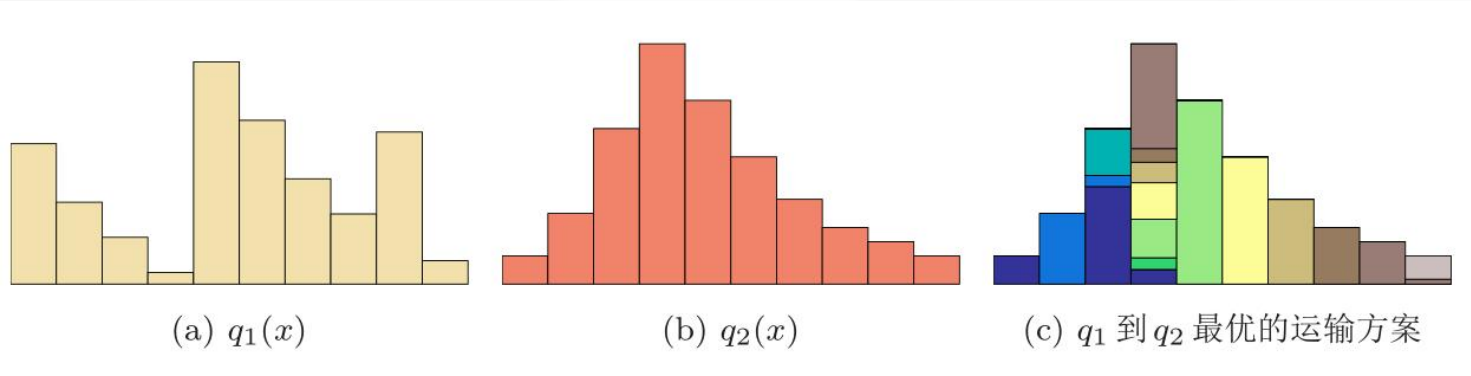
$$W_p(q_1, q_2) = \left(\inf_{\gamma(x,y) \in \Gamma(q_1, q_2)} \mathbb{E}_{(x,y) \sim \gamma(x,y)} [d(x,y)^p] \right)^{\frac{1}{p}}$$

其中$T(q_1,q_2)$是边际分布为q_1,q_2的所有可能的联合分布集合，$d(x,y)$为x和y的距离，比如l_p距离等。

- Wasserstein距离相比于KL散度和JS散度的优势在于：即使两个分布没有重叠或者重叠非常少，Wasserstein距离仍然能反映两个分布的远近。

Wasserstein距离

- 如果将两个分布看作是两个土堆，联合分布γ(x,y)看作是从土堆q_1的位置x到土堆q_2的位置y的搬运土的数量。Wasserstein距离可以理解为搬运土堆的最小工作量，也称为推土机距离(Earth-Mover's Distance, EMD)



Kantorovich-Rubinstein 对偶定理

This formula is highly intractable

$$\mathrm{EMD}(P_r, P_\theta) = \inf_{\gamma \in \Pi(P_r, P_\theta)} \sum_{x,y} \|x - y\| \gamma(x, y) = \inf_{\gamma \in \Pi(P_r, P_\theta)} \mathbb{E}_{(x,y)\sim\gamma} \|x - y\|$$

Kantorovich-Rubinstein Duality

$$\mathrm{EMD}(P_r, P_\theta) = \sup_{\|f\|_L \leq 1} \mathbb{E}_{x\sim P_r} f(x) - \mathbb{E}_{x\sim P_\theta} f(x).$$

1-Lipschitz Constraint

Lipschitz连续函数

数学小知识 | Lipschitz连续函数

在数学中，对于一个实数函数 $f:\mathbb{R}\to\mathbb{R}$，如果满足函数曲线上任意两点连线的斜率一致有界，即任意两点的斜率都小于常数 $K>0$，

$$|f(x_1)-f(x_2)|\leq K|x_1-x_2|, \tag{13.54}$$

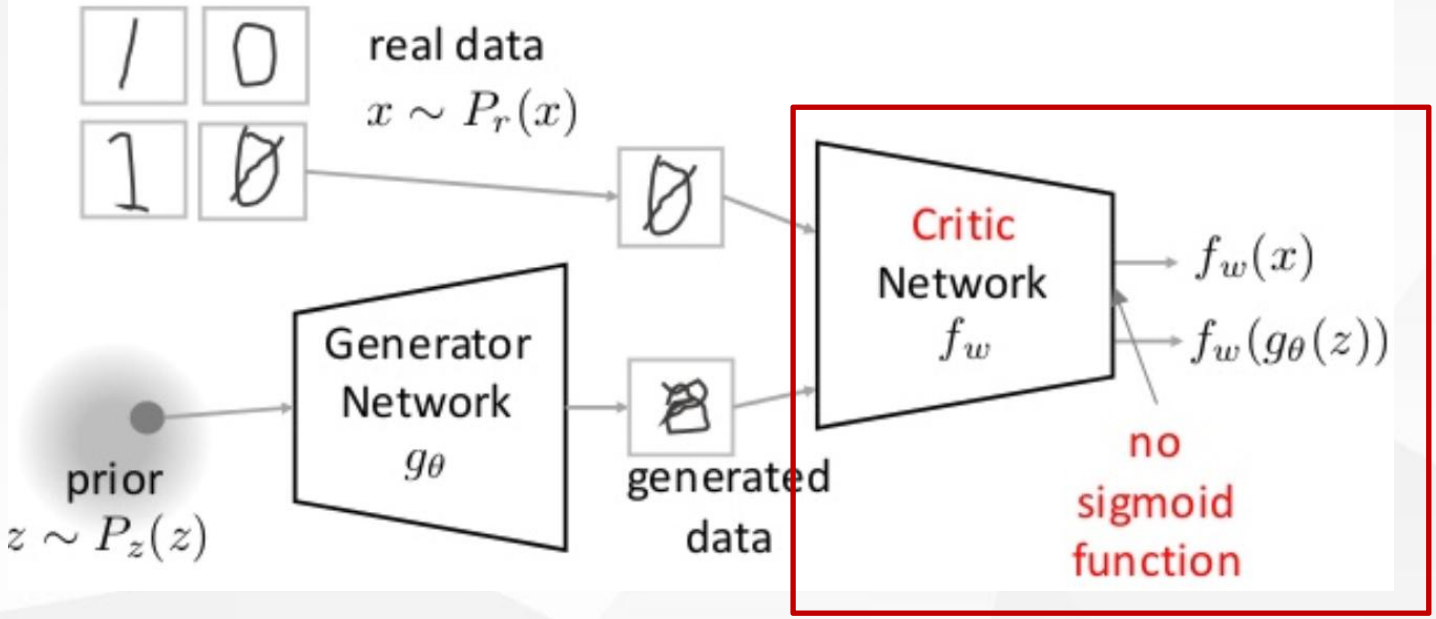
则函数 f 就称为 *K-Lipschitz连续函数*，K 称为Lipschitz常数。

Lipschitz连续要求函数在无限的区间上不能有超过线性的增长。如果一个函数可导，并满足Lipschitz连续，那么导数有界。如果一个函数可导，并且导数有界，那么函数为Lipschitz连续。

Wasserstein GAN

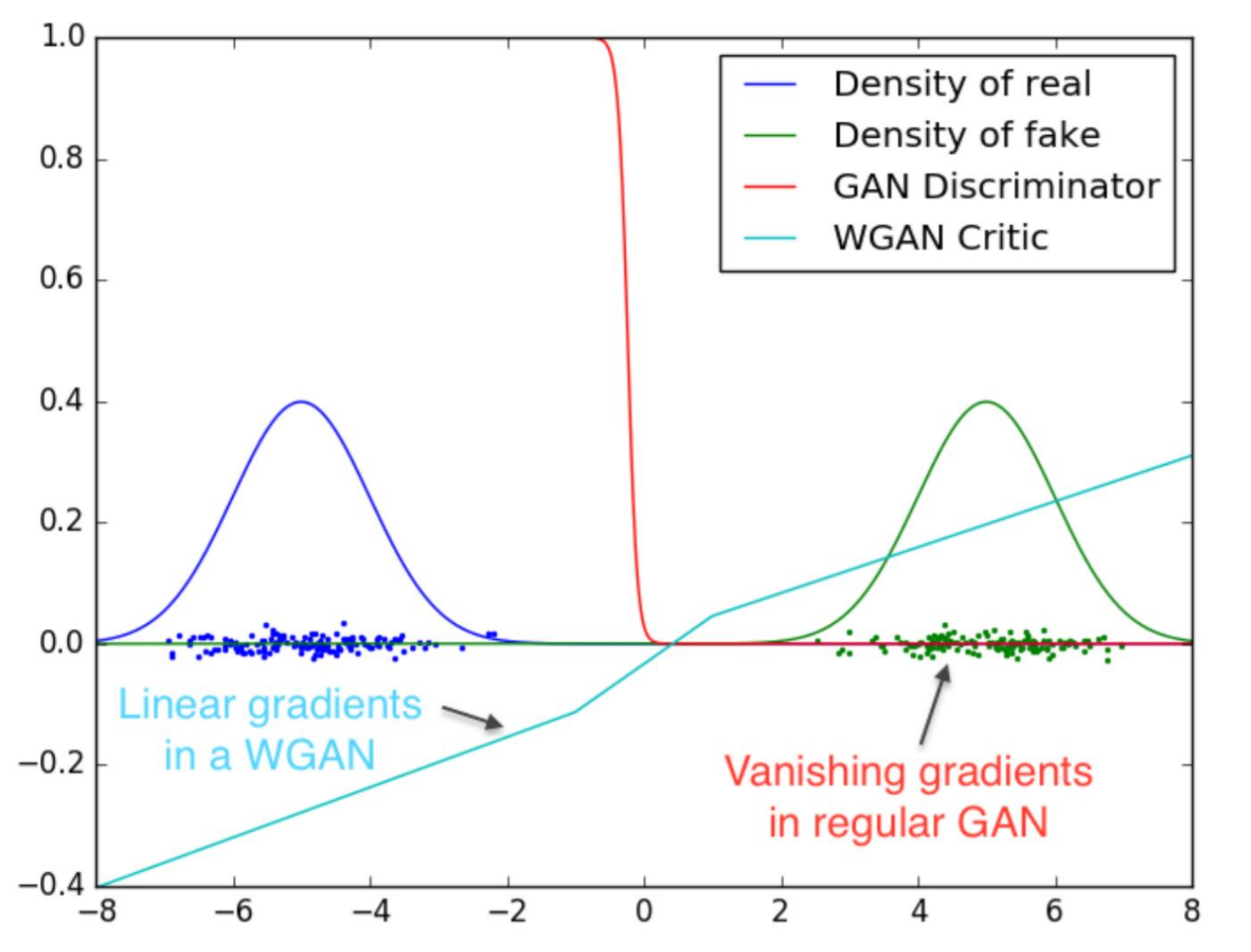
$$\min_{\theta} \max_{w \in [-k,k]^l} \mathbb{E}_{x \sim P_r}[f_w(x)] - \mathbb{E}_{z \sim P_z}[f_w(g_\theta(z))]$$

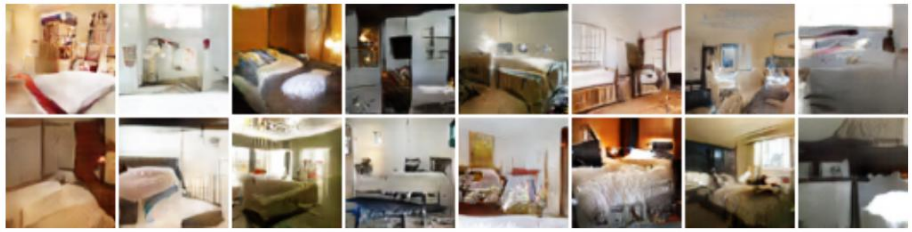
k-Lipschitz Constraint



输出wasserstein距离

梯度问题


Density of real
Density of fake
GAN Discriminator
WGAN Critic
Linear gradients
in a WGAN
Vanishing gradients
in regular GAN
1.0
0.8
0.6
0.4
0.2
0.0
−0.2
−0.4
−8
−6
−4
−2
0
2
4
6
8

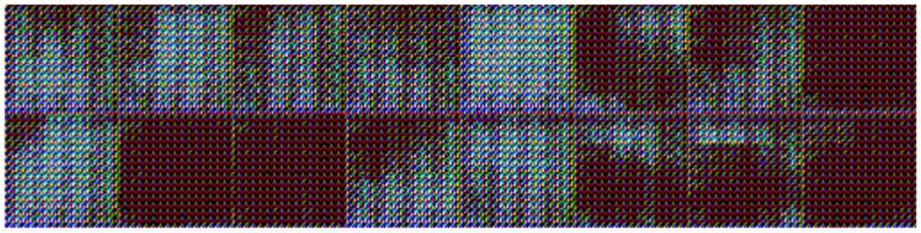


WGAN

DCGAN

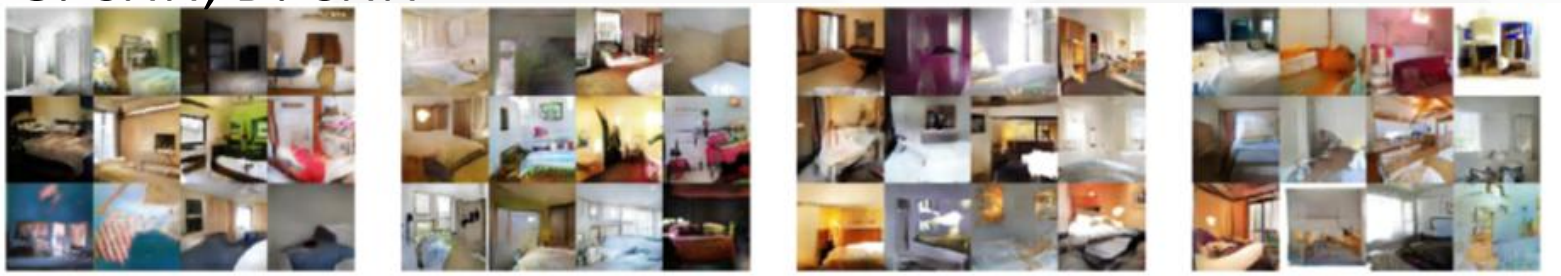
batch normalization

constant number of filters at every layer



DCGAN | LSGAN | Original WGAN | Improved WGAN

G: CNN, D: CNN



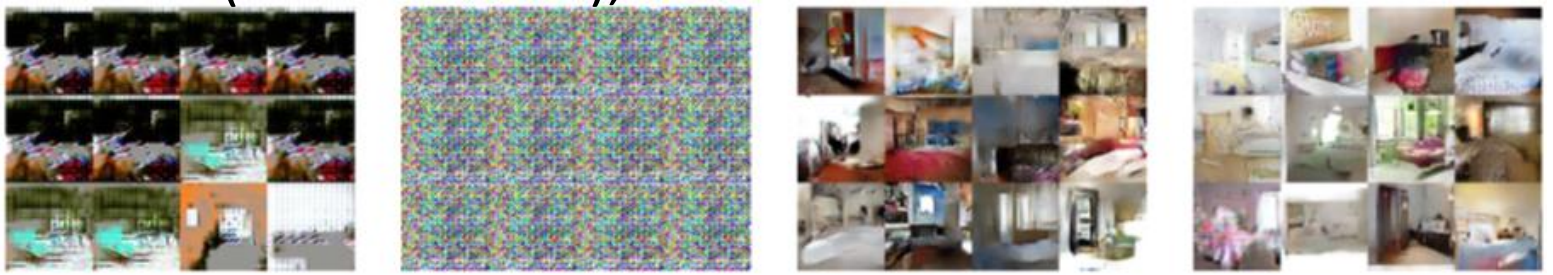
G: CNN (no normalization), D: CNN (no normalization)

G: CNN (tanh), D: CNN(tanh)

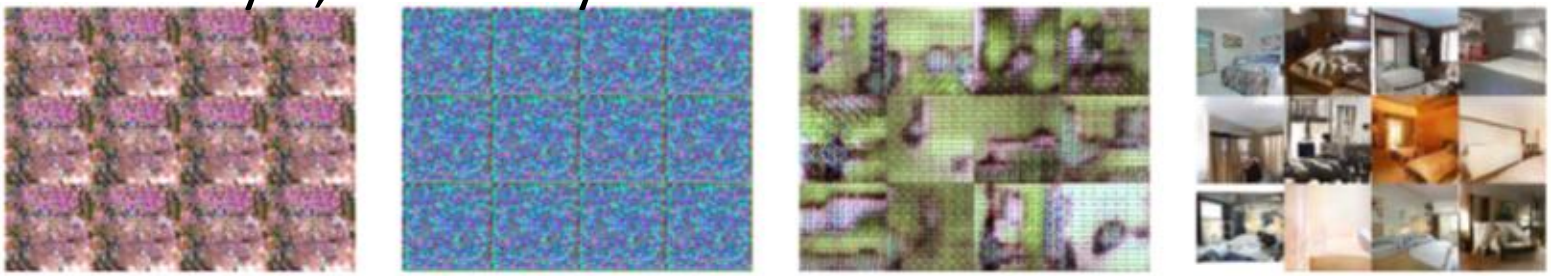
DCGAN | LSGAN | Original WGAN | Improved WGAN

G: MLP, D: CNN

G: CNN (bad structure), D: CNN



G: 101 layer, D: 101 layer



条件生成

- 根据条件针对性的生成数据

$$\begin{bmatrix}0.3\\-0.1\\\vdots\\-0.7\end{bmatrix}\begin{bmatrix}0.1\\-0.1\\\vdots\\0.7\end{bmatrix}\begin{bmatrix}-0.3\\0.1\\\vdots\\0.9\end{bmatrix}$$

In a specific range

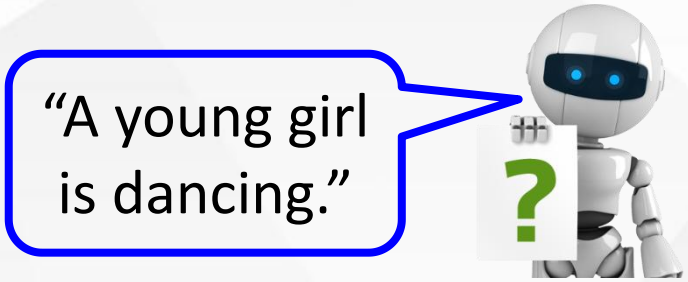
“Girl with red hair and red eyes”

“Girl with yellow ribbon”

条件生成

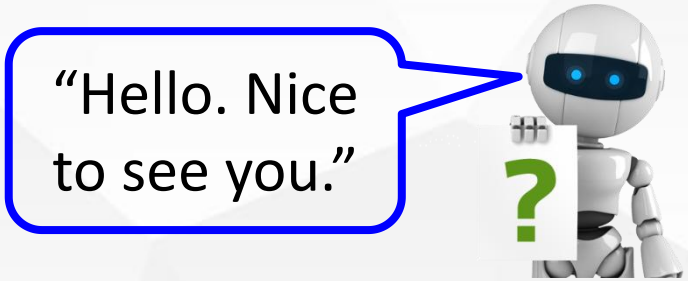
Caption Generation

Given condition:

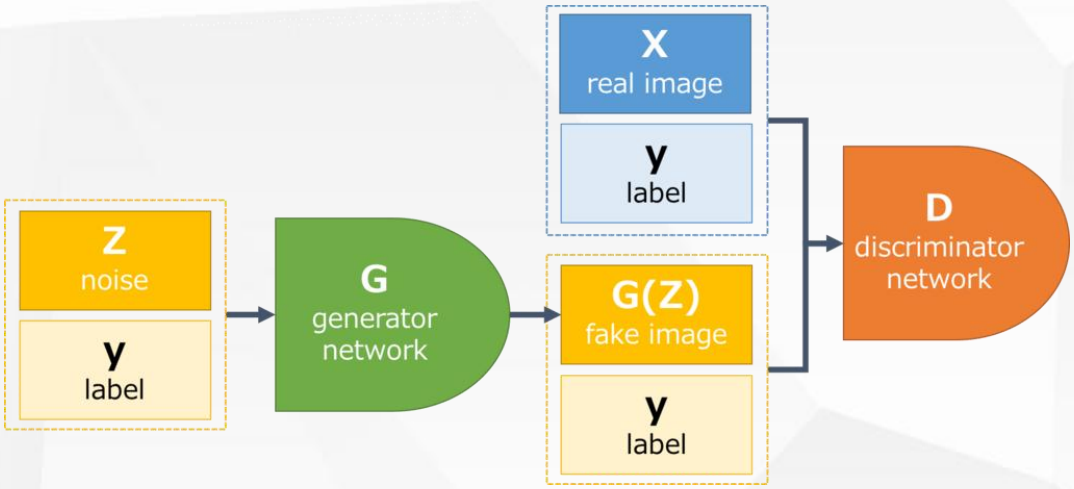


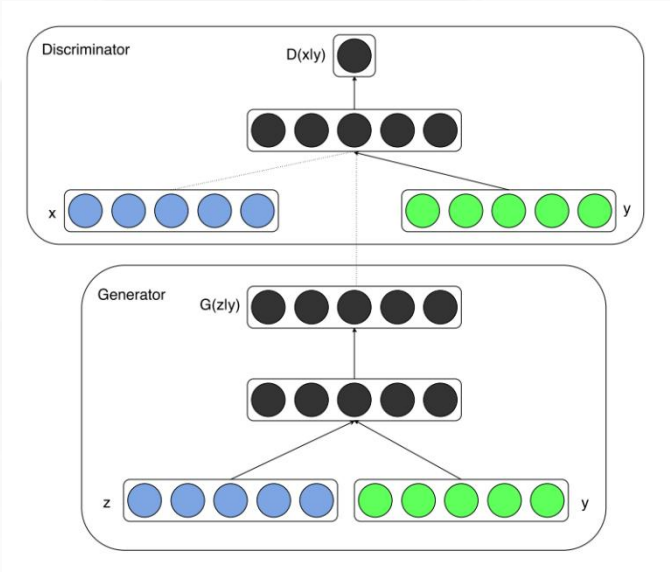
Chat-bot

Given condition:

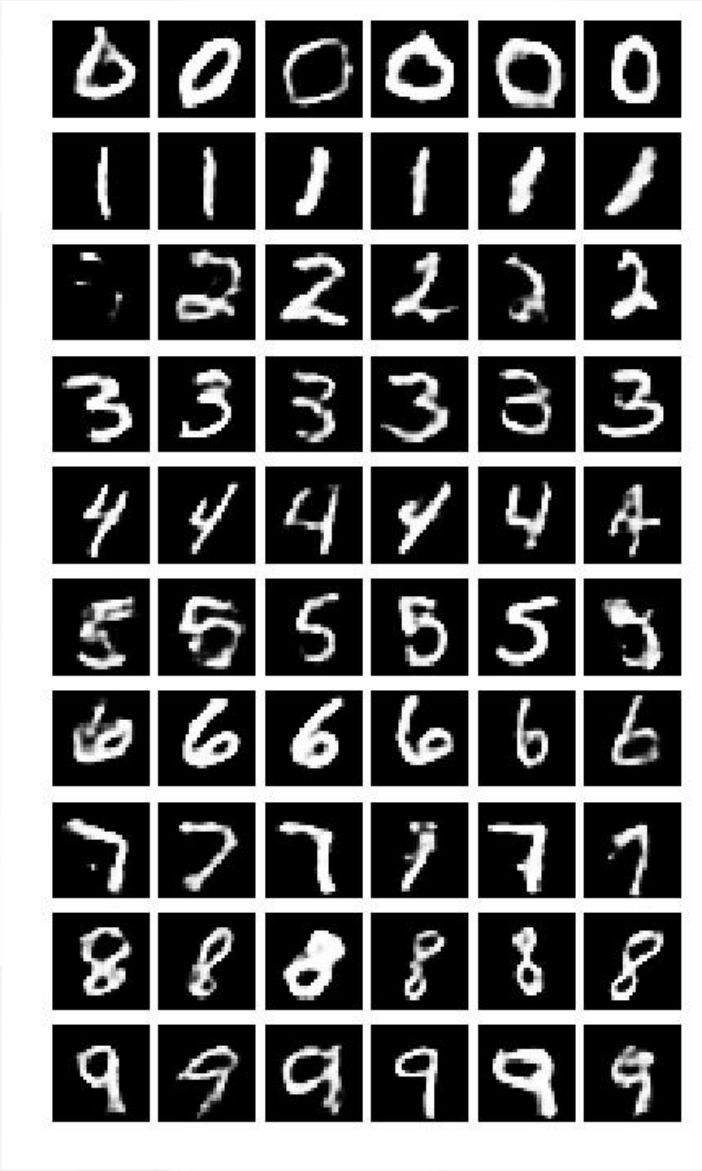


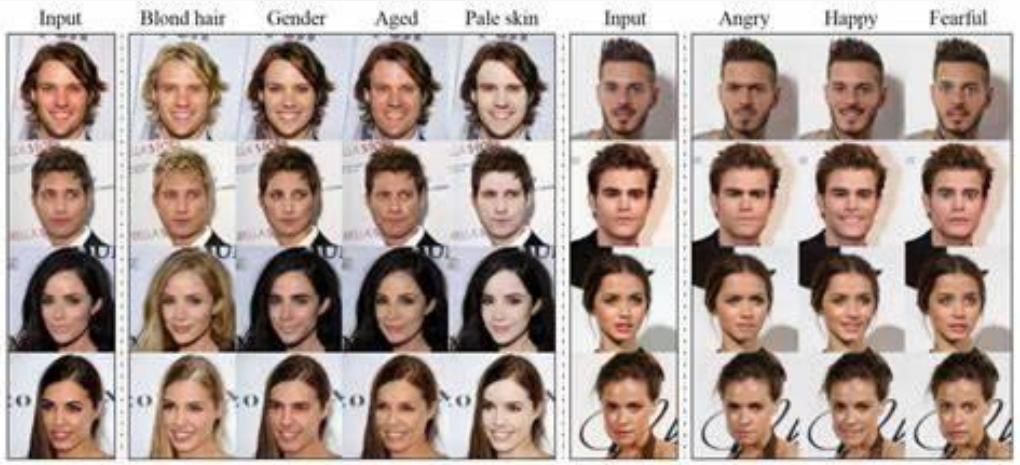
Conditional GAN



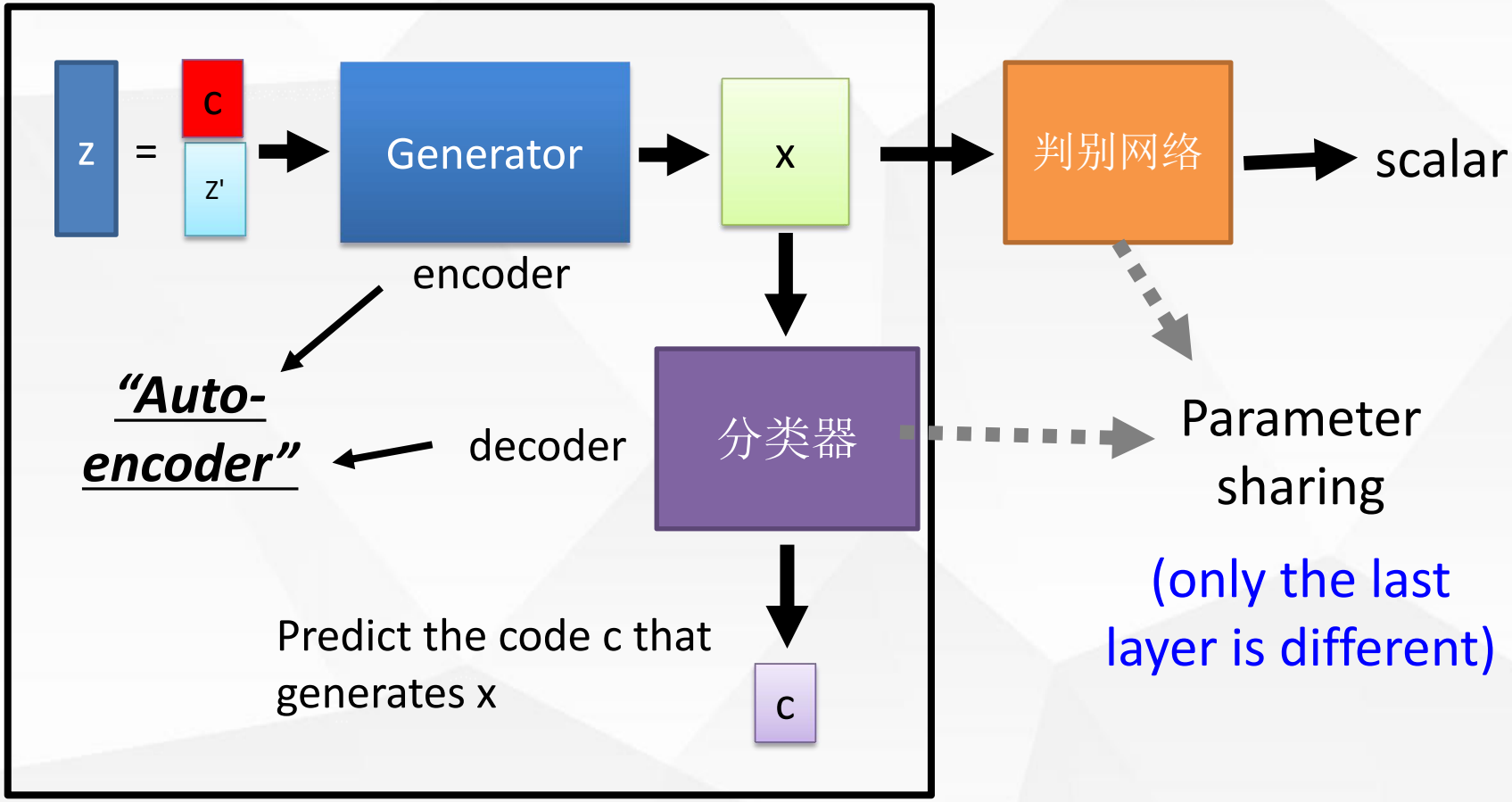


Conditional GAN

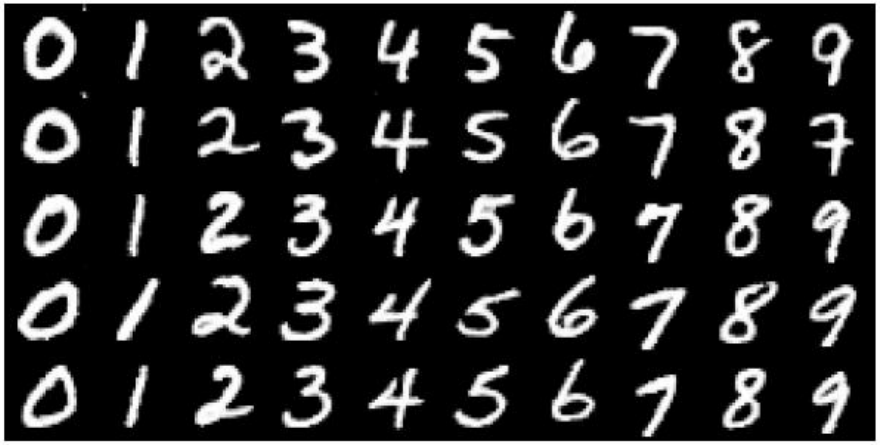




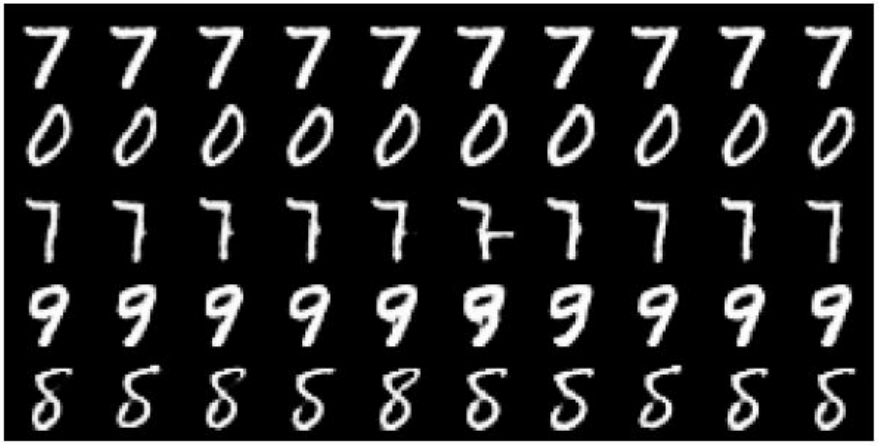
InfoGAN


z
=
c
z'
Generator
x
判别网络
scalar
encoder
"Auto-encoder"
decoder
分类器
Parameter sharing
(only the last layer is different)
Predict the code c that generates x
c

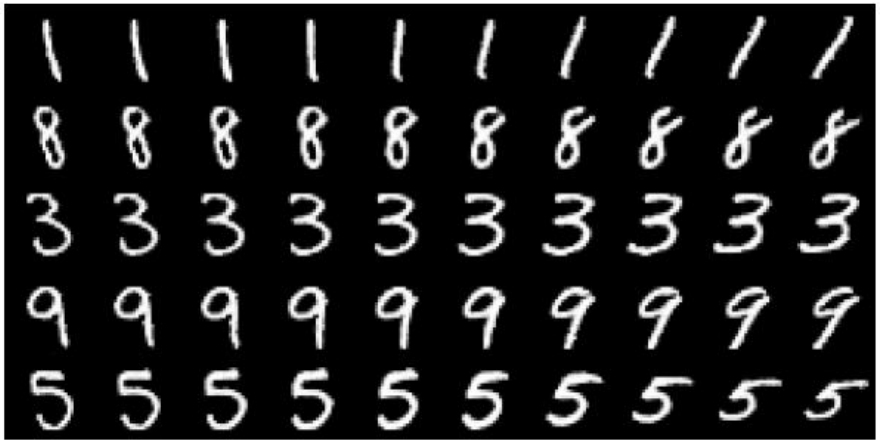
InfoGAN



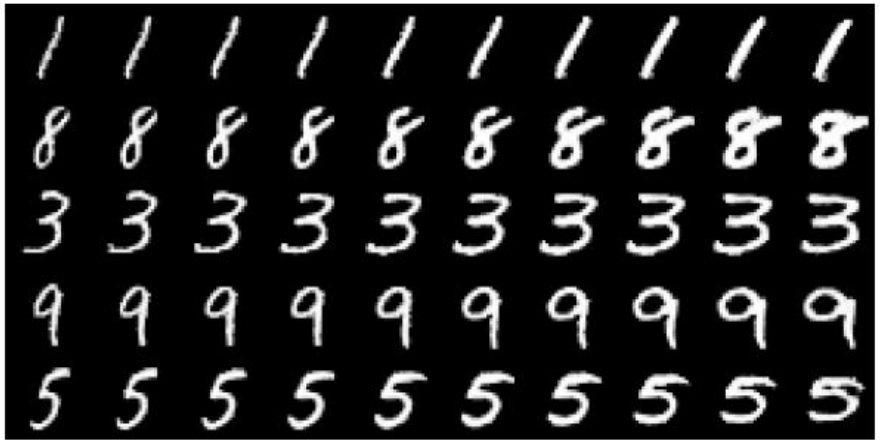
(a) Varying c_1 on InfoGAN (Digit type)



(b) Varying c_1 on regular GAN (No clear meaning)



(c) Varying c_2 from -2 to 2 on InfoGAN (Rotation)



(d) Varying c_3 from -2 to 2 on InfoGAN (Width)

InfoGAN

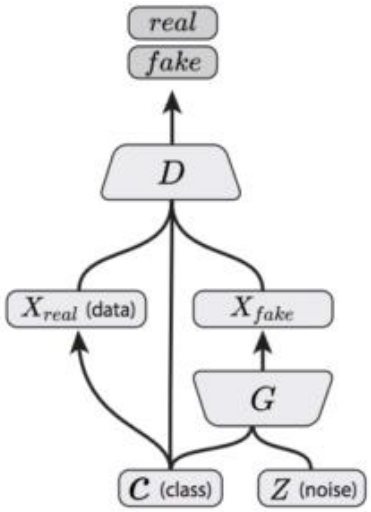
(a) Rotation

(b) Width

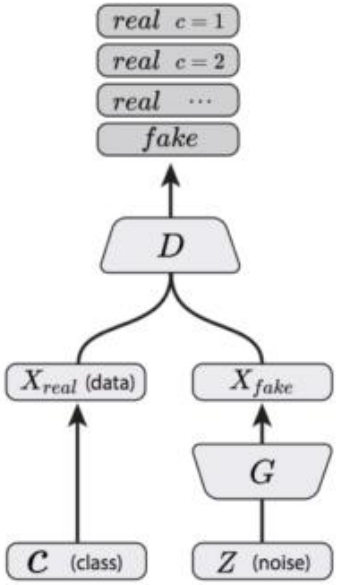
(c) Lighting

(d) Wide or Narrow

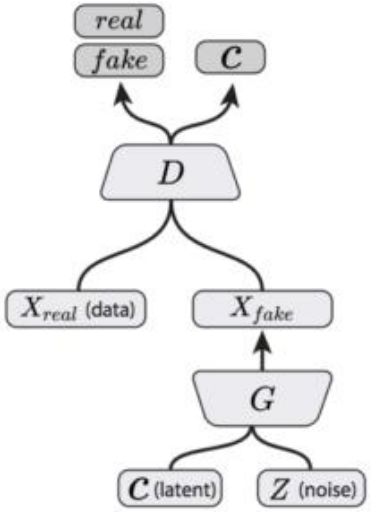
AC-GAN



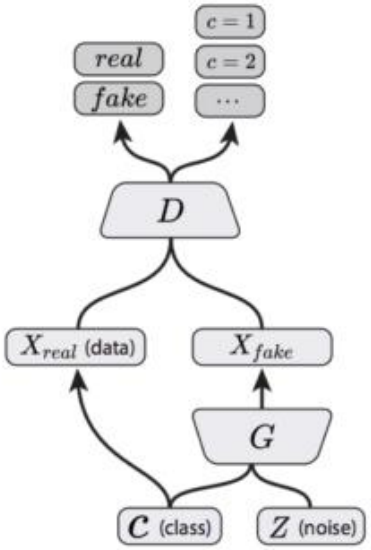
Conditional GAN
(Mirza & Osindero, 2014)



Semi-Supervised GAN
(Odena, 2016; Salimans, et al., 2016)

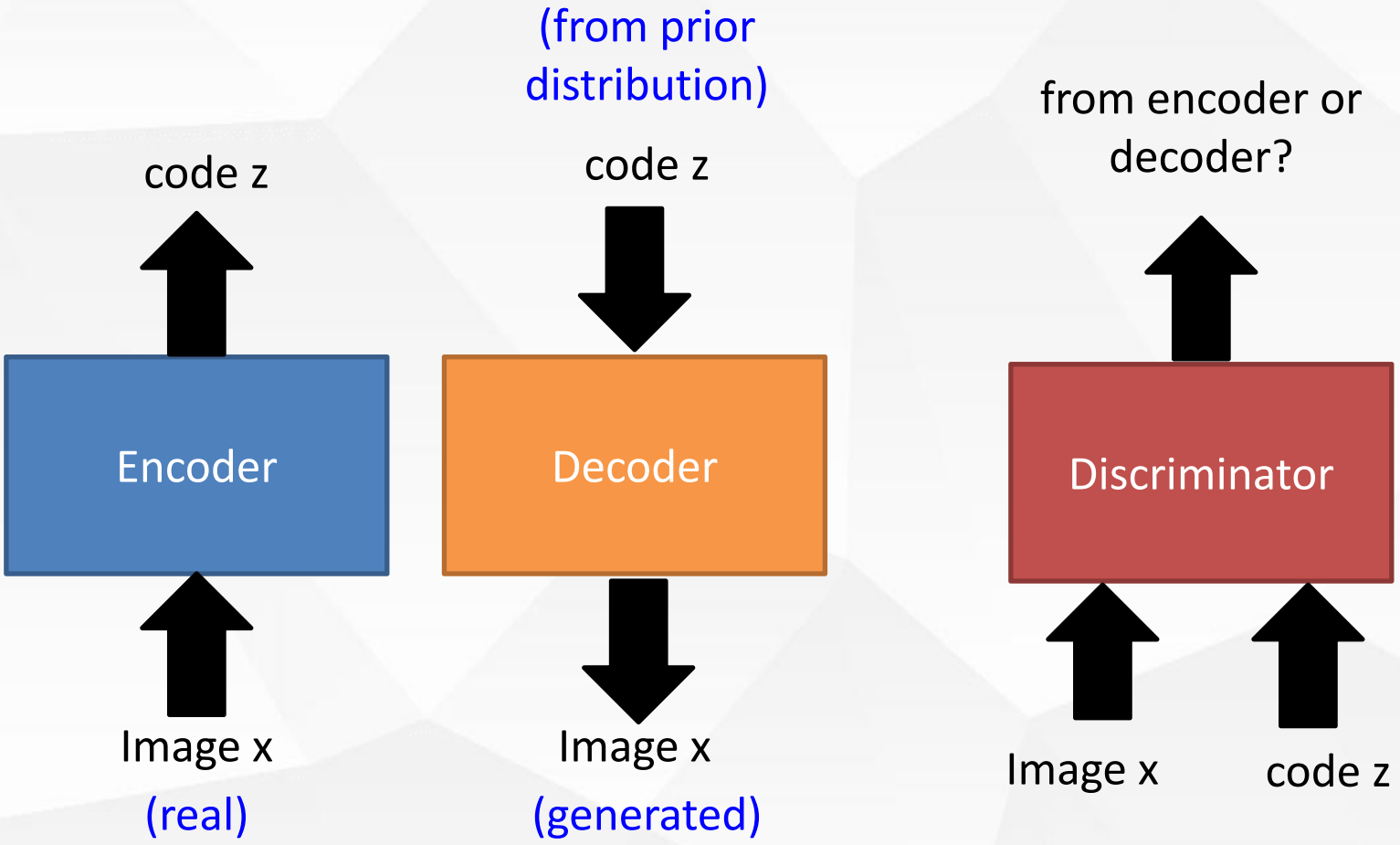


InfoGAN
(Chen, et al., 2016)



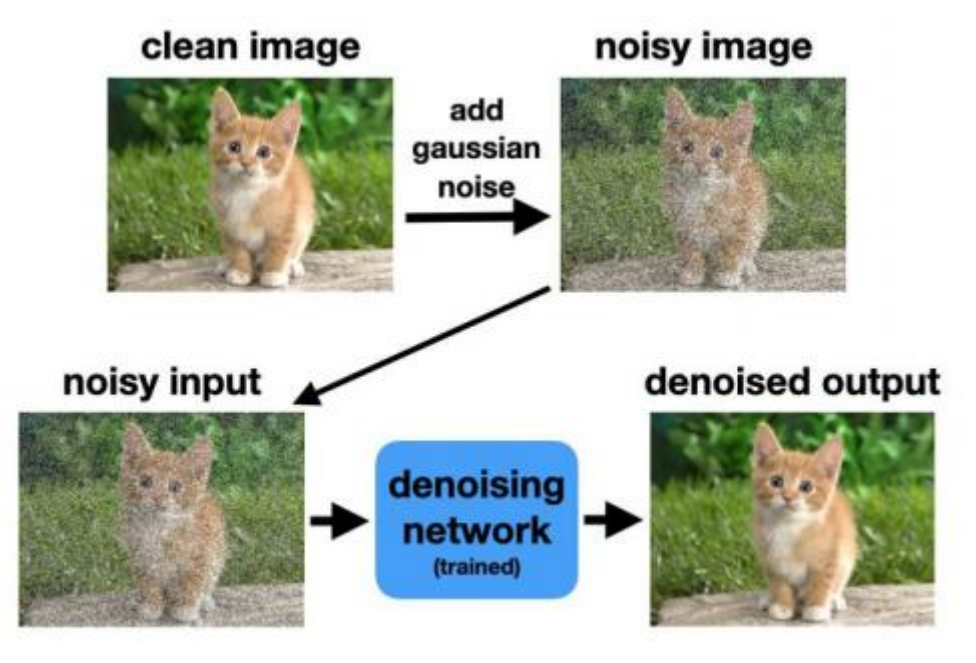
AC-GAN
(Present Work)

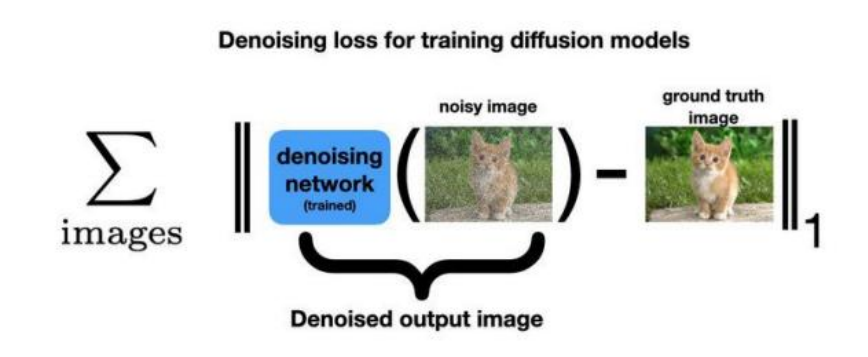
BiGAN


(from prior distribution)
code z
code z
Encoder
Decoder
Image x
(real)
Image x
(generated)
from encoder or decoder?
Discriminator
Image x
code z

Diffusion Model

Diffusion最初应用：图像去噪

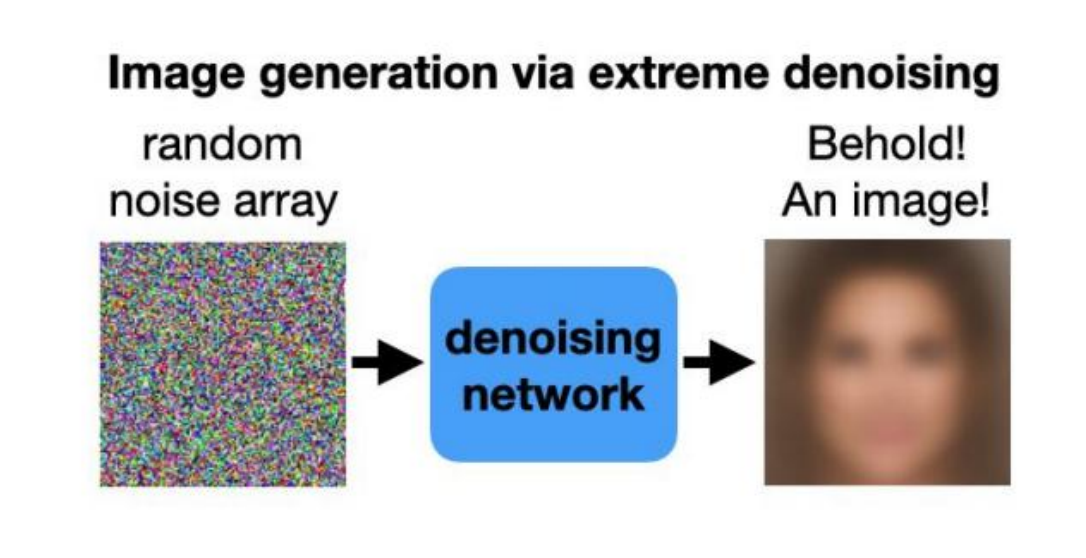




Diffusion Model

Diffusion最初应用：图像去噪

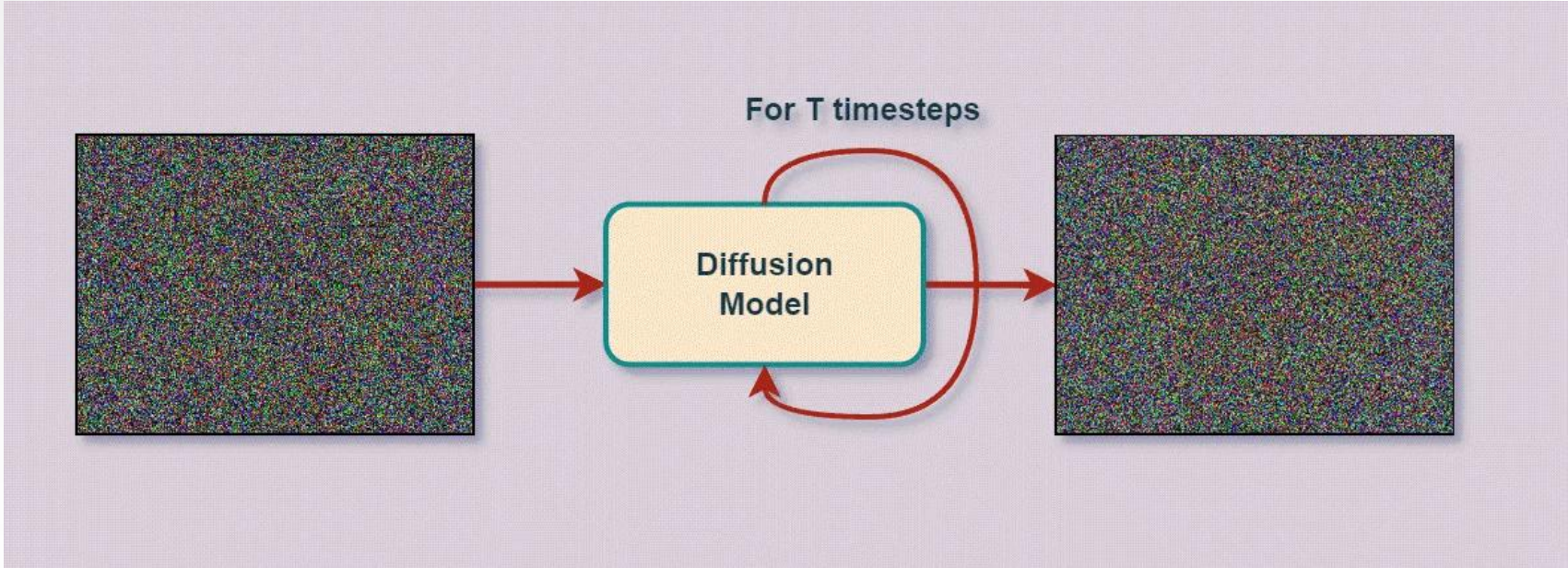
这是不是意味着给定一张完全是噪声的图像也可以通过“去噪”恢复出完整图像



Diffusion Model

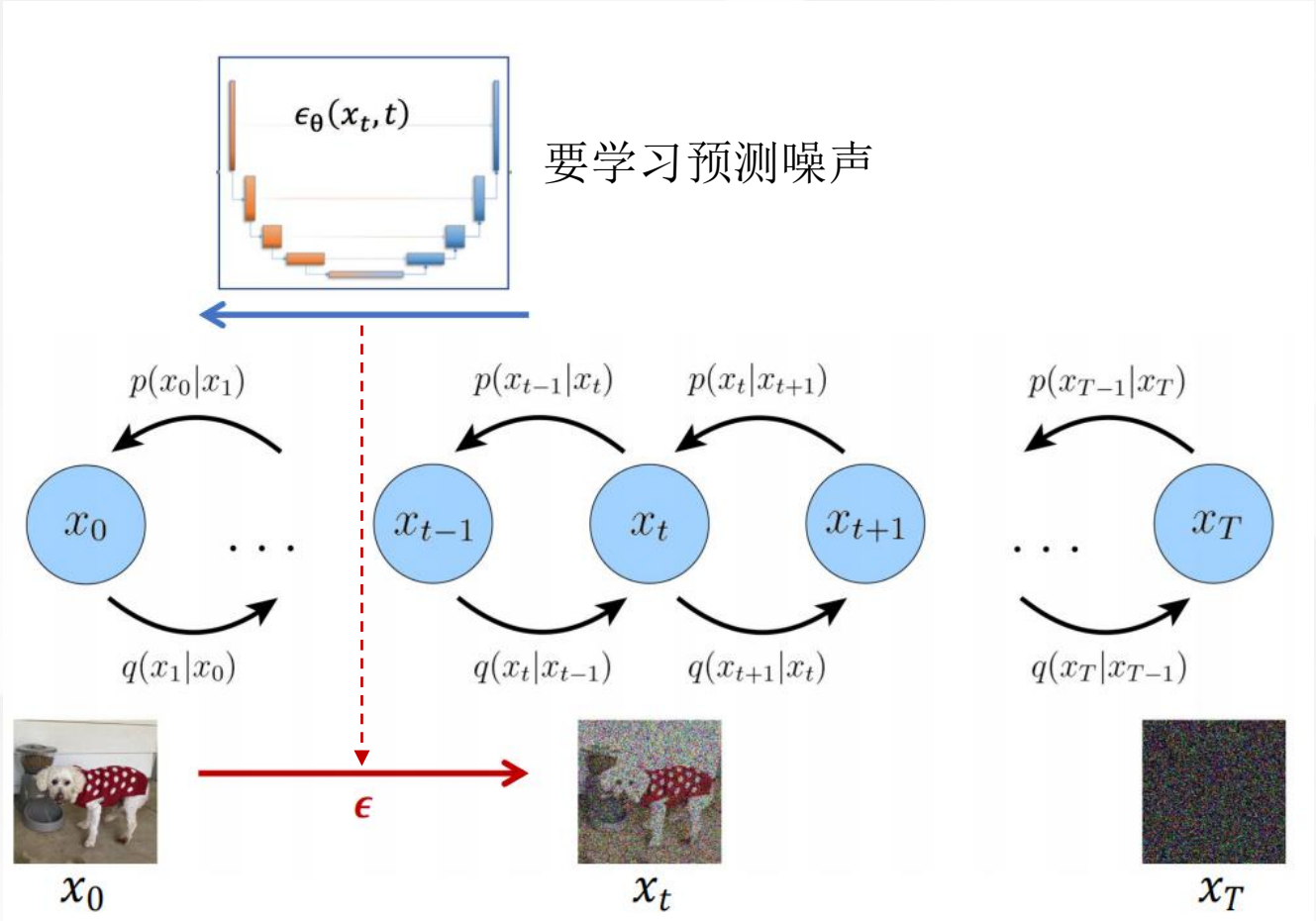
Diffusion最初应用：图像去噪

这是不是意味着给定一张完全是噪声的图像也可以通过“去噪”恢复出完整图像

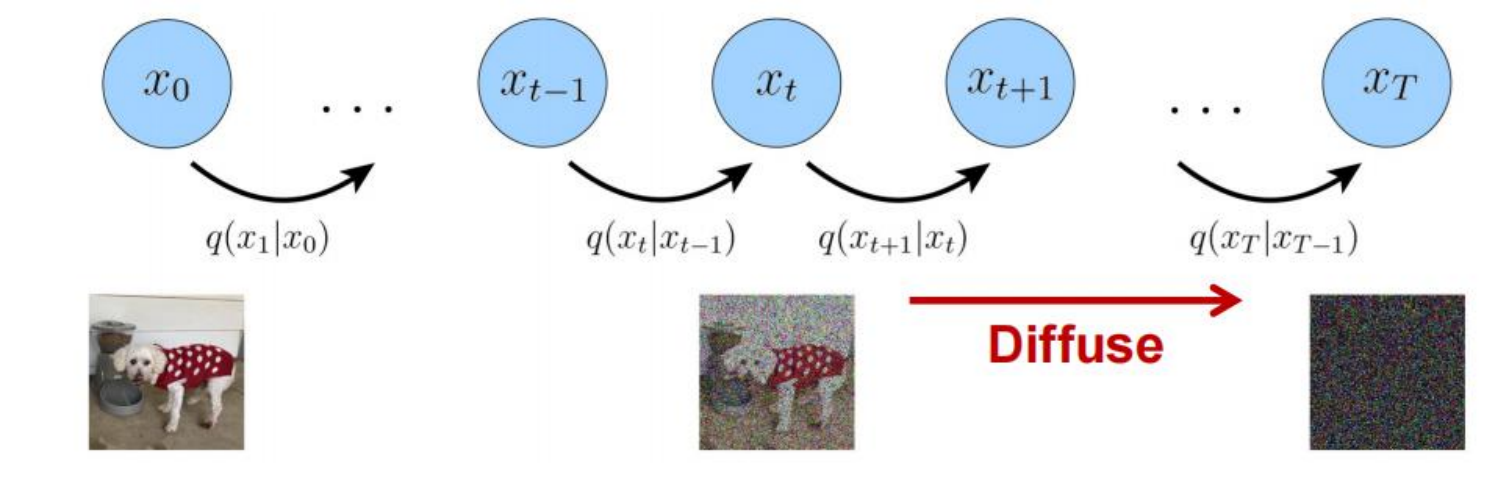


Diffusion Model

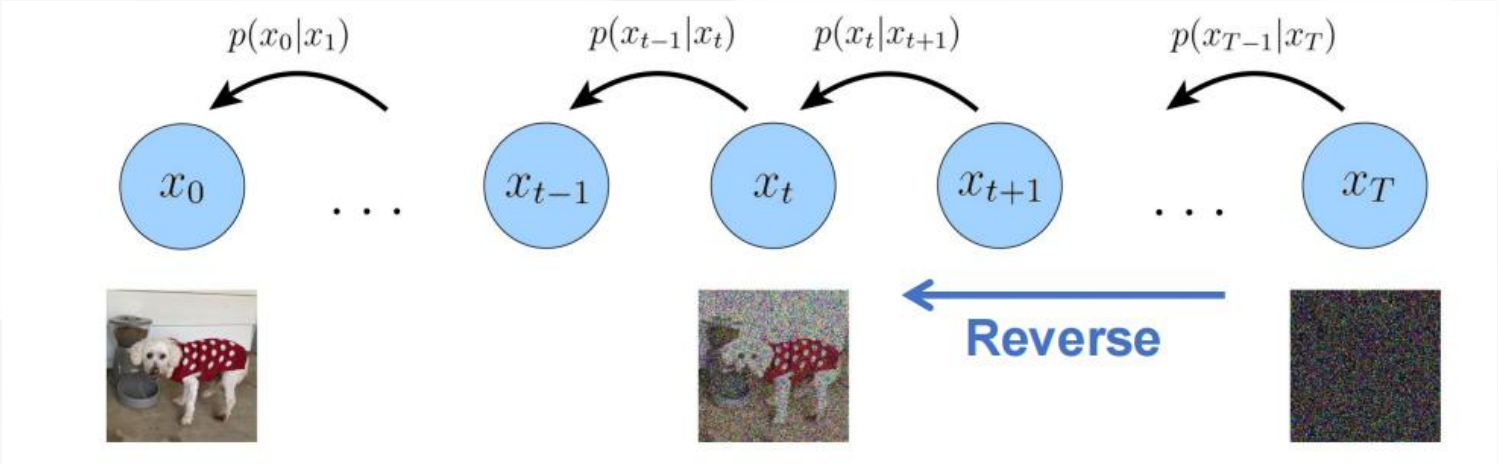
Diffusion做图像生成的动机：逐步加入特定噪声，再逐步去噪



Diffusion Model



随机给定一个噪声，噪声预测网络预测扩散阶段每一步添加的噪声，实现反向去噪



Diffusion Model

Algorithm 1 Training

1: **repeat**
2: $\mathbf{x}_0 \sim q(\mathbf{x}_0)$
3: $t \sim \text{Uniform}(\{1, \dots, T\})$
4: $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
5: Take gradient descent step on
$$\nabla_\theta \left\| \boldsymbol{\epsilon} - \boldsymbol{\epsilon}_\theta(\sqrt{\bar{\alpha}_t}\mathbf{x}_0 + \sqrt{1-\bar{\alpha}_t}\boldsymbol{\epsilon}, t) \right\|^2$$
6: **until** converged

基于训练集训练噪声预测网络

Algorithm 2 Sampling

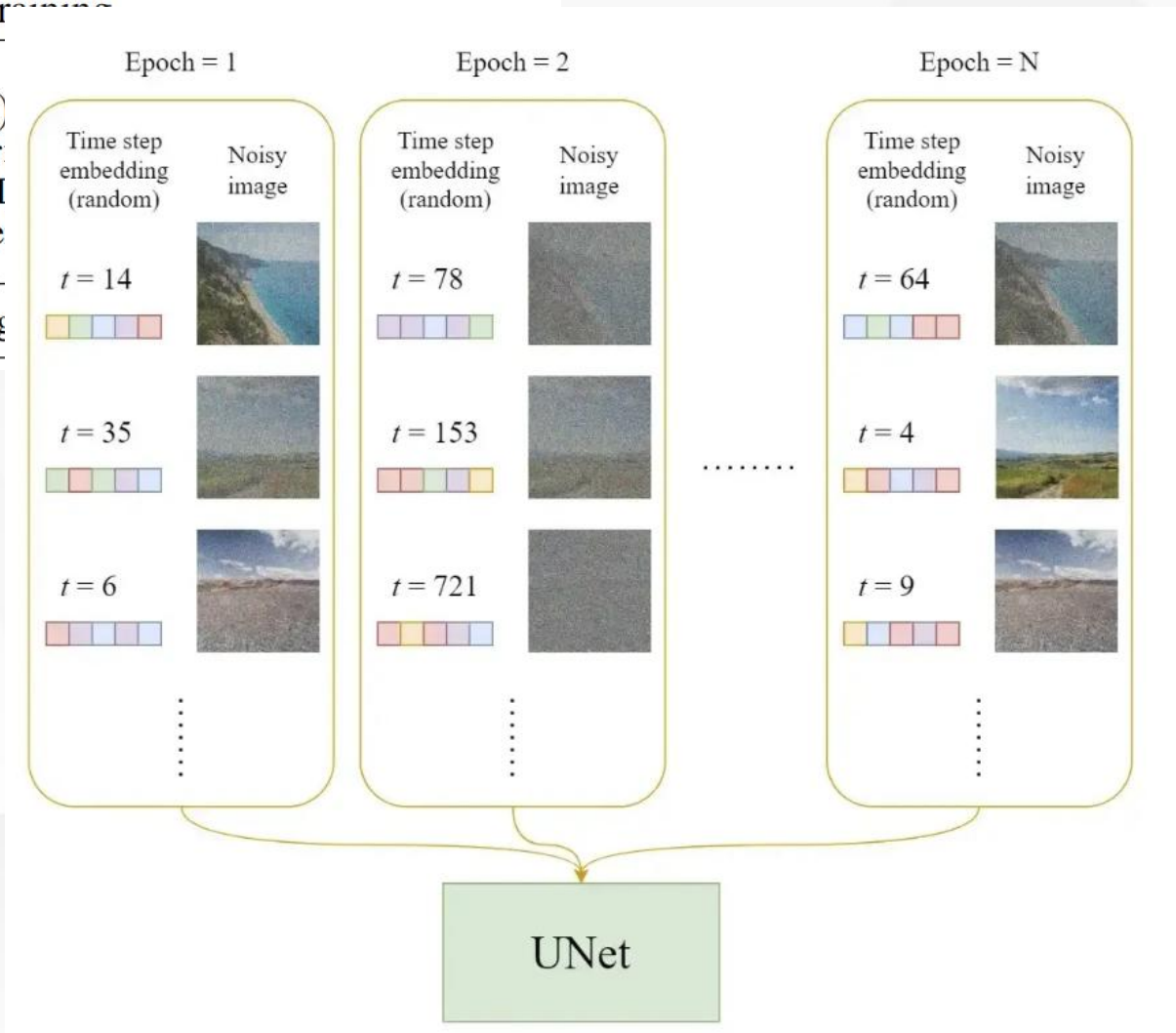
1: $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
2: **for** $t = T, \dots, 1$ **do**
3: $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ if $t > 1$, else $\mathbf{z} = \mathbf{0}$
4: $\mathbf{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left(\mathbf{x}_t - \frac{1-\alpha_t}{\sqrt{1-\bar{\alpha}_t}} \boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t) \right) + \sigma_t \mathbf{z}$
5: **end for**
6: **return** $\mathbf{x}_0$

给定一个起始点，预测其每一步“被添加的”噪声，实现去噪

Diffusion Model

Algorithm 1 Tr

1: **repeat**
2: $\mathbf{x}_0 \sim q(\mathbf{x}_0)$
3: $t \sim$ Unifor
4: $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}$
5: Take gradie
$\nabla_\theta \left\| \boldsymbol{\epsilon} -$
6: **until** converg



Diffus

Algorithm 1 T

1: **repeat**
2: $\mathbf{x}_0 \sim q(\mathbf{x}_0$
3: $t \sim$ Unifo
4: $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0},$
5: Take gradi
$\nabla_\theta \|\boldsymbol{\epsilon}$
6: **until** conve

1. Randomly select a time step & encode it

2. Add noise to image

$$x_t = \sqrt{\bar{a}_t}\, x_0 + \sqrt{1-\bar{a}_t}\, \varepsilon$$

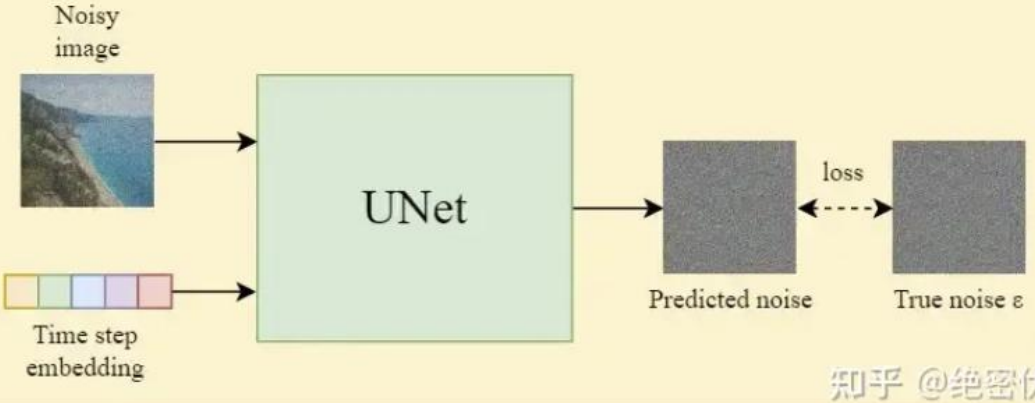
Adjust the amount of noise according to the time step t

$$\varepsilon \sim \mathcal{N}(0,1)$$

$$\alpha_t = 1 - \beta_t$$

$$\bar{\alpha}_t = \prod_{i=1}^{t} \alpha_i$$

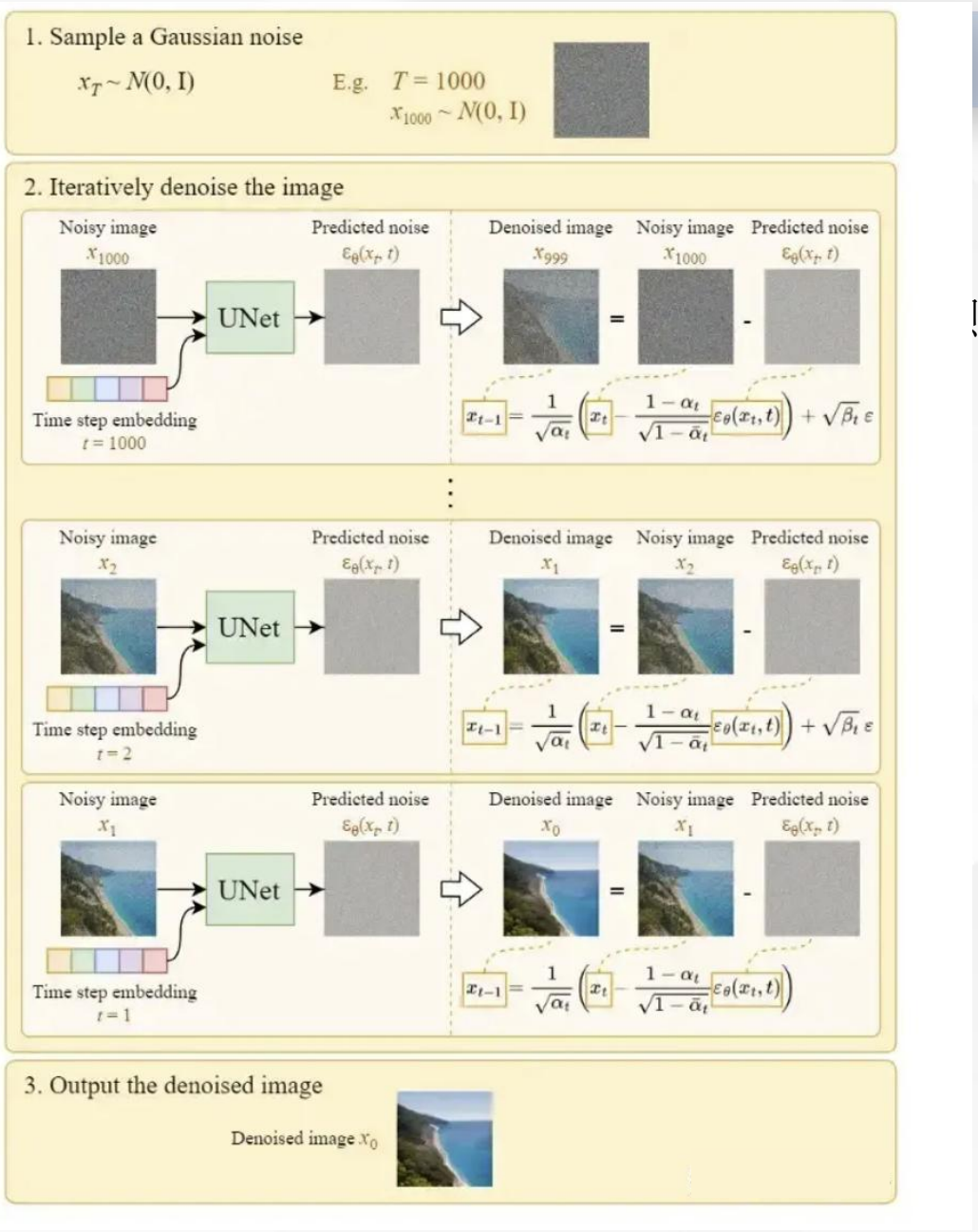
3. Train the UNet



Diffusi

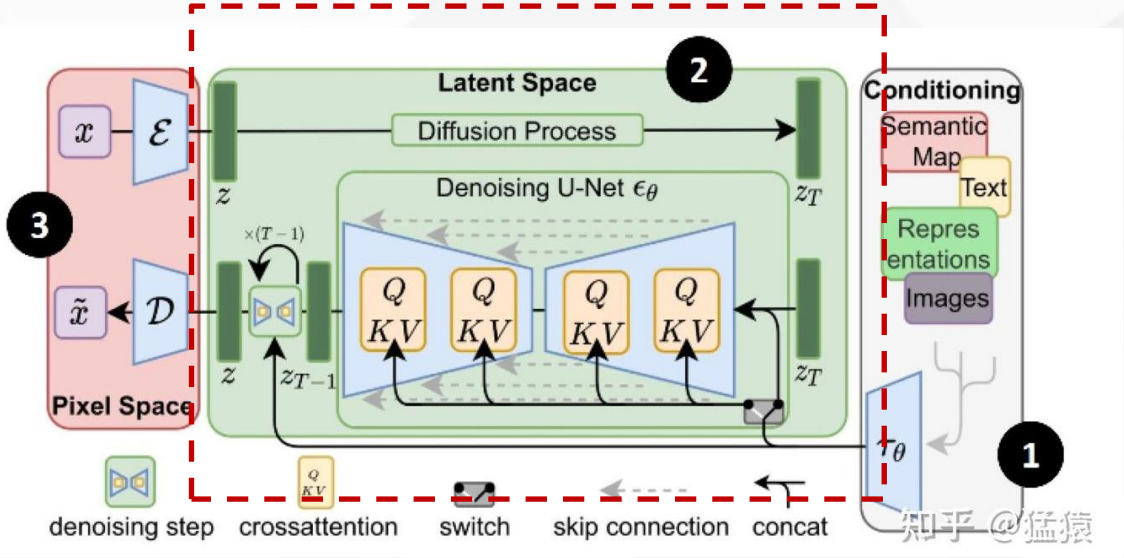
Algorithm 2 Samp

1: $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
2: **for** $t = T, \ldots, 1$
3: $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ if
4: $\mathbf{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left($
5: **end for**
6: **return** $\mathbf{x}_0$



Diffusion Model

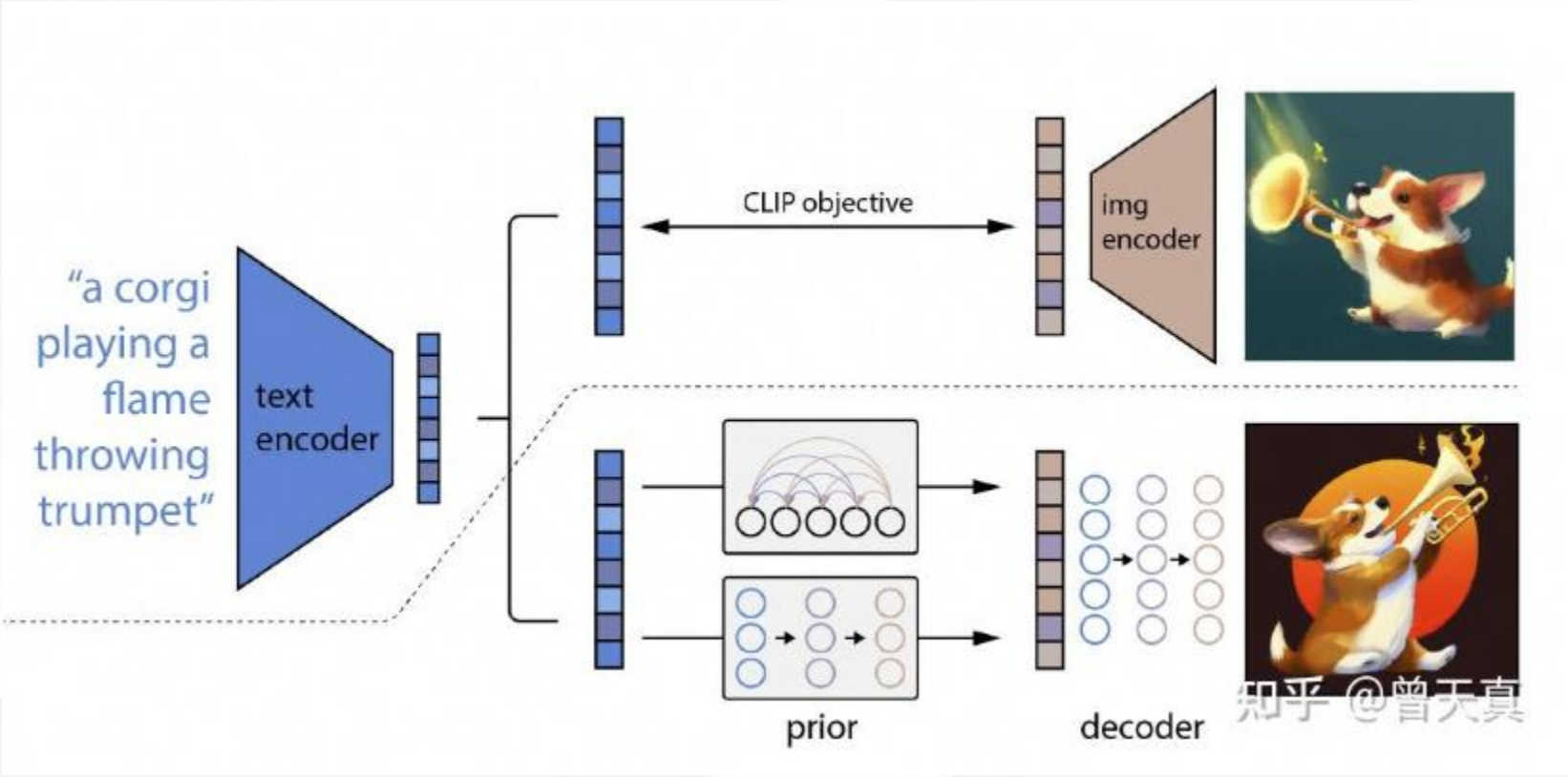
Stable Diffusion:目前最为经典的Diffusion Model



已有的Diffusion Model在像素级别进行加噪去噪时间复杂度太高

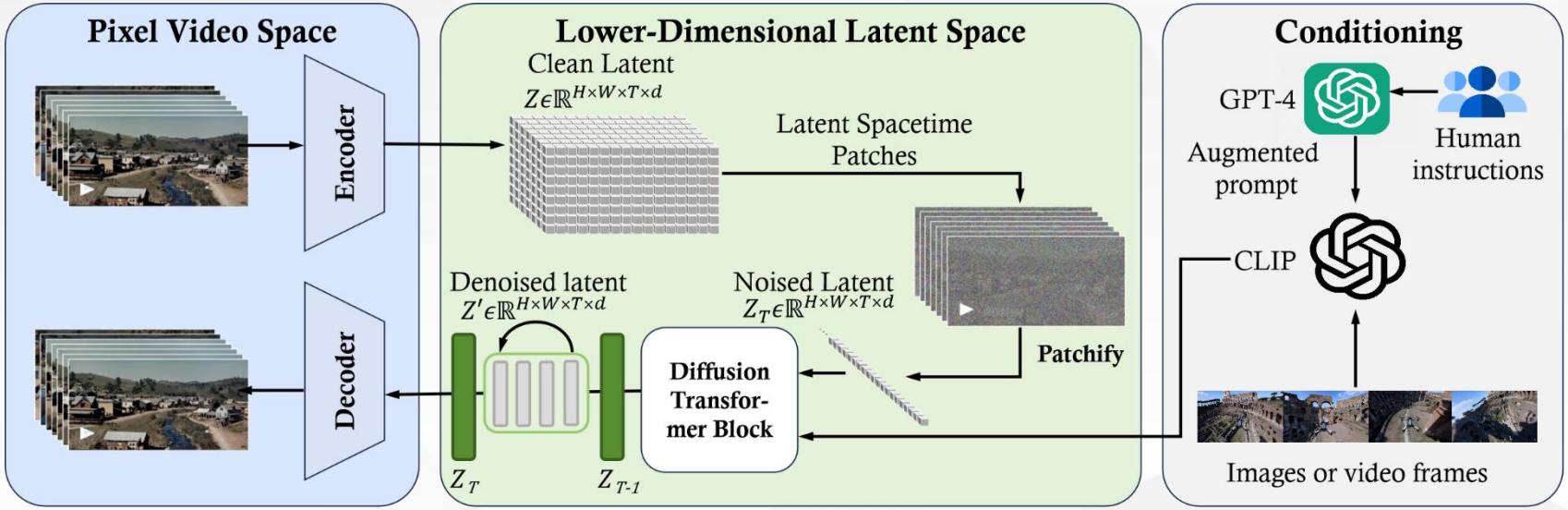
Diffusion Model

Stable Diffusion广泛应用：DALLE



Diffusion Model

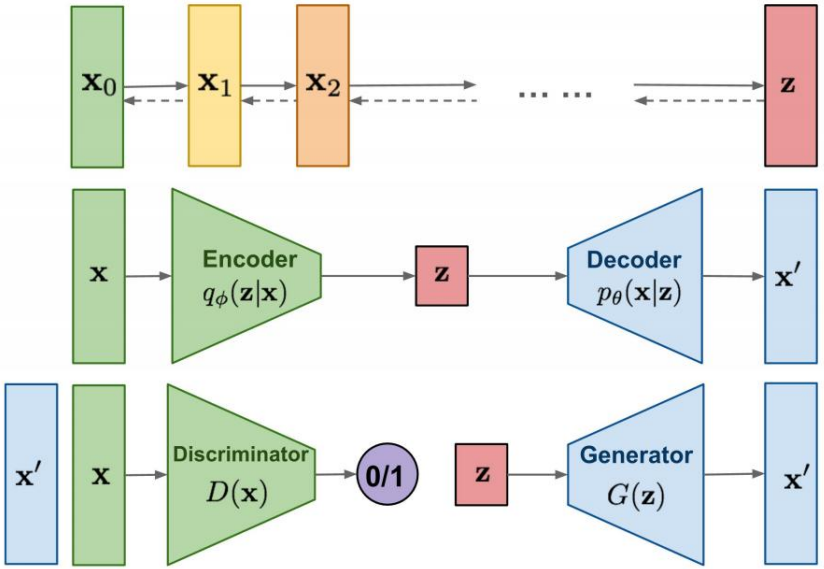
Stable Diffusion广泛应用：Sora



Diffusion Model

主流生成模型的比较：

- **Diffusion Model**
  加噪 + 去噪
- **VAE**
  隐变量 + 变分推断
- **GAN**
  Discriminator学习度量标准
  训练不稳定、mode collapse



谢谢！

THANK YOU FOR LISTENING